
Workgroup: Network Working Group
Internet-Draft: draft-ietf-dconn-domainconnect-04
:
Published: 7 October 2026
Intended Status: Standards Track
Expires: 10 April 2027
Authors: P. Kowalik A. Blinn J. Kolker S. Kerola
DENIC eG GoDaddy Inc. Cloudflare, Inc.

Domain Connect Protocol - DNS provisioning between Services and DNS Providers

Abstract

This document provides specification of the Domain Connect Protocol that was built to support DNS configuration provisioning between Service Providers (hosting, social, email, hardware, etc.) and DNS Providers.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 10 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	5
2. Use Cases	5
2.1. Primary Use Cases	5
2.2. Use Cases out of scope	6
4. Protocol design	12
4.1. Templates	12
4.2. Trust Model	12
4.3. Protocol Versioning and Extensibility	12
5. Protocol Flows Overview	14
5.1. General information	14
5.2. The Synchronous Flow	14
6. Domain Connect Objects and Templates	17
6.1. Template Definition	17
6.1.1. Template Property Extensibility	22
6.2. Template Record	23
6.2.1. Template Record Extensibility	29
6.3. Template Considerations	30
6.3.1. Template Scope	30
6.3.2. Sub Domains	30
6.3.3. Variable Scope Minimization	30
6.4. Public Key Publication	31
7. DNS Provider Discovery	33
7.1. "_domainconnect" Resource Record	33
7.1.1. Deployment considerations	33
7.2. Settings End-Point	34
7.2.1. Settings Property Extensibility	37

8. Applying Domain Connect	38
8.1. Endpoints	38
8.2. Query Supported Template	38
8.3. Synchronous Flow	40
8.3.1. Apply Template URL	40
8.3.2. Parameters/properties	40
8.3.3. Template Apply Request	45
8.3.4. Signature Verification	46
8.3.5. Template Apply Response	46
8.3.6. Template Apply Error Response	47
8.4. Verification of Changes	48
9. Resolving Template Variables	48
9.1. Variables	48
9.1.1. Variable Syntax	48
9.1.2. Special and Built-In Variables	48
9.2. Variable substitution	49
9.2.1. Input Values	49
9.2.2. Processing	49
9.3. Host Name Rendering	50
9.3.1. Input Values	50
9.3.2. Processing	50
9.3.3. Examples of host processing	50
9.4. SPF Record Merging	51
10. Template Application	52
10.1. Template State in DNS Providers system	52
10.2. Steps to Apply a Template	53
10.3. Group Filtering	54
10.3.1. Group Filtering and Template State	54
10.4. Conflict Detection	55
10.4.1. Conflict Detection Special Handling	55

10.5. Calculating Conflict Resolution	56
10.6. User Authorization of Changes	56
10.7. Template Application	57
10.8. Examples	57
11. Security Considerations	57
11.1. Template Variable Phishing	57
11.2. Untrusted DNS Provider Discovery Records	58
11.3. Underscored Host Names	58
11.4. Must-Understand Markers and Pre-Existing Deployments	59
11.5. Template Onboarding	59
12. IANA Considerations	60
12.1. Registration Procedure for Domain Connect Registries	60
12.2. Domain Connect Fully Specified Record Types Registry	61
12.3. Domain Connect Settings Properties Registry	63
12.4. Domain Connect Template Properties Registry	64
12.5. Domain Connect Protocol-Level Record Fields Registry	66
12.6. Guidance to the DNS Resource Record (RR) TYPEs Registry	67
12.7. Underscore "_domainconnect" DNS Node Name	68
12.8. Media Type Registration for application/domainconnect-template+json	68
Implementation Status	69
Acknowledgements	70
Change History	70
Normative References	74
Informative References	76
Appendix A. Examples	77
A.1. Example Template	77
A.2. Example Records: Single static host record	77
A.3. Example Records: Single variable host record for A	78
A.4. Example Records: Generic Record Type CAA	78
A.5. Example: Vendor-Specific Extensions and Critical Markers	79

A.6. Example: Template application to DNS Zone and Conflict Resolution	80
A.7. Example: SPF Record Merging	81
Authors' Addresses	83

1. Introduction

Connecting a domain name to a service should be a simple and straightforward process. However, historically, users have faced a complex and often frustrating task involving manual DNS configuration. Traditional methods are unreliable, require deep technical knowledge of DNS, and result in outdated and confusing instructions from Service Providers. This leads to user frustration, support overhead, and abandoned setups.

To address these challenges, Domain Connect offers a streamlined and automated solution. It empowers Service Providers to easily enable their services to work with user domains, simplifying both DNS provider discovery and DNS configuration. By abstracting away the complexities of manual DNS management through user-friendly web interactions, standard authentication, and template-based configurations, Domain Connect significantly improves the user experience.

2. Use Cases

2.1. Primary Use Cases

The following use cases illustrate the wide range of applications where Domain Connect simplifies and automates DNS configuration, from basic service onboarding to complex, dynamic DNS management scenarios.

- **SaaS Provider with One-Off DNS Configuration:** A Software as a Service (SaaS) Provider offering functionality with an option to assign own domain name, such as web hosting or email, can utilize Domain Connect to streamline the process of configuring DNS records for their customers. This automation eliminates the need for manual configuration and simplifies the onboarding experience for users.
- **SaaS Provider with Multi-Step DNS Configuration:** Some SaaS Providers may require a multi-step DNS configuration process. For example, a service might require initial verification of domain ownership through a TXT record, followed by the creation of CNAME records for different subdomains. Domain Connect can handle such scenarios by utilizing record groups ("groupId") to apply parts of a template in separate interactions, each with explicit user consent.
- **On-Premise Service with Publicly Accessible DNS Service:** An on-premise service, such as a local network device or server, can also benefit from Domain Connect if it utilizes a publicly accessible DNS service. By leveraging Domain Connect, the service can automatically update

DNS records as needed, ensuring that the service remains accessible through its domain name.

- **Tool or Service with Regularly Updated DNS Entries:** A tool or service that requires regular updates to DNS entries, such as a dynamic DNS service or a DNS-based load balancer, can use Domain Connect to automate the process.
- **Packaged Software Provider:** A packaged software provider, whether open-source or proprietary, can integrate Domain Connect into their installation and configuration process. This allows the software to automatically configure necessary DNS records during installation, simplifying the setup process for users. However, if the software is installed on a private network with a private DNS service, it might not be directly compatible with Domain Connect, unless the DNS service provides Domain Connect endpoints accessible to the installation process.

2.2. Use Cases out of scope

While Domain Connect offers significant advantages in automating DNS configuration, it's important to recognize scenarios where it might not be the ideal solution:

- **Automation or CI/CD Pipelines:** Domain Connect is primarily designed for user-driven DNS configuration, where an end user grants consent and applies changes. Automating this process within CI/CD pipelines or other automated workflows is outside the scope of this specification.
- **Private/Enterprise DNS with Public SaaS Providers:** Domain Connect relies on public DNS records and endpoints to facilitate discovery and configuration. If a private or enterprise DNS service is used, it might not be directly compatible with Domain Connect, unless the DNS service provides publicly accessible Domain Connect endpoints.

3. Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

The Terms like "Registrar", "Authoritative server", "Zone", "Zone Apex" or "Sub Domain" are used as defined in [RFC9499].

This specification uses the Augmented Backus-Naur Form (ABNF) notation of [RFC5234]. The following ABNF rules are imported from the normative references [RFC5234].

ALPHA	=	%x41-5A / %x61-7A	;	A-Z / a-z
DIGIT	=	%x30-39	;	0-9

The following definitions and rules are imported normatively from other documents.

"a-label": "A-label" as defined in Section 2.3.2.1 of [RFC5890].

"u-label": "U-label" as defined in Section 2.3.2.1 of [RFC5890].

"ldh-label": "NR-LDH Label" as defined in Section 2.3.2.2. of [RFC5890].

"domain-name": "Internationalized Domain Name" as defined in Section 2.3.2.3. of [RFC5890].

"state": "state" as defined in Appendix A. Section A5 of [RFC6749].

"redirect_uri": "redirect_uri" as defined in Appendix A, Section A.6 of [RFC6749].

"unicode-assignable": "Unicode Assignables" as defined in Section 4.3 of [RFC9839].

"JSON number": "number" as defined in Section 6 of [RFC8259].

"srv-rdata": SRV RDATA fields as defined in Section 3.1 of [RFC2782].

"service-name": "Service Name" as defined in Section 5.1 of [RFC6335].

"presentation format": "Presentation Format" as defined in Section 1 of [RFC9499].

The following additional syntax rules are defined in this document and used normatively throughout.

"dc-name-char": any Unicode Assignable code point allowed in "domain-name" except "%" and "@"

The following additional ABNF rules are defined in this document and used normatively throughout.

```
; ---- Identifier rules ----

dc-id          = 1*63( ALPHA / DIGIT / "-" / "_" / "." )
                ; non-empty token, max 63 octets;
                ; used for providerId, serviceId, groupId,
                ; instanceId

dc-id-list     = dc-id *( "," dc-id )
                ; comma-separated list of dc-id values;
                ; used for groupId parameter

dc-host-list   = domain-name *( *SP "," *SP domain-name )
                ; comma-separated list of domain names

dc-apply-param = dc-predefined-param / dc-dot-param / dc-vnd-param
                ; any apply-parameter name

dc-predefined-param =
    "providerId" / "serviceId" / "domain" / "host"
    / "redirect_uri" / "state" / "providerName"
    / "serviceName" / "groupId" / "sig" / "key"
```

```
        / "crit"
        / "client_id" / "response_type" / "scope"
        ; parameter names defined by this specification or
        ; carried over from the client-request calls of the
        ; asynchronous flow of [DC-SPEC]

dc-crit-list = dc-apply-param *( "," dc-apply-param )
        ; comma-separated list of apply parameter names;
        ; used for the crit parameter

dc-dot-param = "dc." 1*ALPHA
        ; "dc." tree, reserved for standards-track extensions

dc-vnd-param = "vnd." 1*ALPHA *( "." 1*ALPHA )
        ; "vnd." tree, reserved for vendor/private use

dc-template-property = dc-unprefixed-property / dc-vnd-property
        ; any template-definition property name

dc-unprefixed-property = ( ALPHA / DIGIT ) *( ALPHA / DIGIT )
        ; reserved for this specification and for
        ; standards-track extensions; managed by the
        ; IANA Domain Connect Template Properties registry

dc-vnd-property = "vnd." 1*ALPHA *( "." 1*ALPHA )
        ; "vnd." tree for vendor/private template
        ; properties

dc-record-field = dc-unprefixed-property / dc-vnd-property
        ; any template record field name

dc-settings-property = dc-unprefixed-property / dc-vnd-property
        ; any settings-response property name

; ---- Other parameter rules ----

dc-sig = 1*( ALPHA / DIGIT / "+" / "/" / "=" )
        ; standard base64 alphabet per RFC 4648 Section 4

dc-underscore-label = "_" 1*( ALPHA / DIGIT / "-" )
        ; RFC 8552 underscore-prefixed DNS label,
        ; max 63 octets total

dc-key-label = a-label / dc-underscore-label
        ; plain ACE label or RFC 8552 underscore label;
        ; used for the key parameter

dc-pubkey-domain = *( dc-underscore-label "." ) domain-name
        ; zero or more underscore labels prepended
        ; to an IDNA domain name, per RFC 8552
        ; user for syncPubKeyDomain

dc-display-name = 1*255unicode-assignable
        ; non-empty string of at most 255 Unicode
        ; Assignable code points per RFC 9839 Section 4.3;
        ; used for providerName and serviceName

dc-description-text = 0*2048unicode-assignable
```

```
        ; a string of at most 2048 Unicode
        ; Assignable code points per RFC 9839 Section 4.3;
        ; used for description and variableDescription

dc-version      = %x31-39 *DIGIT
        ; positive integer, no leading zeros,
        ; no fraction, no exponent;
        ; a strict subset of JSON number (RFC 8259
        ; Section 6); used for template version

; ---- Variable expression rules ----

variable-expression = named-variable / template-apex-var
        ; parameterized expression in a template field

named-variable    = "%" variable-name "%"
        ; %-delimited named variable

template-apex-var = "@"
        ; zone-apex / fqdn shorthand; MUST appear alone
        ; in host/name or pointsTo/data fields

variable-name      = 1*( ALPHA / DIGIT / "-" / "_" )
        ; identifier of a template variable

; ---- Template record field rules ----

dc-record-type    = dc-type-name / dc-rfc3597-type
        ; a registered RR TYPE mnemonic (covering
        ; Fully Specified, Computed, and Generic types
        ; alike), or the RFC 3597 unknown-type form.

dc-type-name      = "A"
        / ( UALPHA *( UALPHA / DIGIT / "-" )
        ( UALPHA / DIGIT ) )
        ; RR TYPE mnemonic per RFC 6895:
        ; [A-Z][A-Z0-9-]*[A-Z0-9], i.e. starts with an
        ; uppercase letter and ends with a letter or
        ; digit; "A" is grandfathered as a single-letter
        ; mnemonic. MUST NOT be of the reserved forms
        ; "TYPE"<n> or "CLASS"<n> (<<RFC3597>>, <<RFC6895>>).

dc-rfc3597-type   = "TYPE" 1*DIGIT
        ; unknown type per RFC 3597

UALPHA            = %x41-5A
        ; A-Z (uppercase letter)

dc-essential      = "Always" / "OnApply"
        ; default is "Always"

dc-txt-conflict-mode = "None" / "All" / "Prefix"
        ; default is "None"

dc-conflict-prefix = 1*(%x21-7E)
        ; non-empty printable ASCII (U+0021..U+007E);
        ; no variable expressions permitted
```

```
; -- property value field --

dc-prop-value = *( dc-prop-char / dc-rdata-escape )
                ; DNS presentation format (RFC 9499);

dc-prop-char  = %x20-7E
                ; space and all printable ASCII (U+0020..U+007E);

dc-rdata-escape = "\" ( 3DIGIT / %x21-7E )
                ; RFC 1035 Section 5.1 octet escape:
                ; "\DDD" three decimal digits, or
                ; "\" followed by any printable non-digit character

; -- host / name / pointsTo / target fields --

dc-host-tmpl   = ""
                ; empty string
                / template-apex-var
                ; "@" alone
                / 1*( named-variable / dc-name-char )
                ; mix of literal chars and %-variables

dc-pointsto-tmpl = template-apex-var
                ; "@" alone
                / 1*( dc-name-char / named-variable )
                ; mix of literal chars and %-variables

dc-pointsto-nat-tmpl = 1*( dc-name-char / named-variable )
                ; mix of literal chars and %-variables

; -- integer fields with optional variable --

dc-ttl-tmpl    = 1*DIGIT / named-variable
                ; constant seconds, or a sole %-variable

dc-ttl-value   = 1*DIGIT
                ; non-negative integer seconds, resolved

dc-uint16-tmpl = ( %x30-39 *4DIGIT ) / named-variable
                ; 0-65535 literal, or a sole %-variable

dc-uint16-value = %x30-39 *4DIGIT
                ; resolved non-negative integer, max 5 digits,
                ; semantic range 0-65535

; -- SRV-specific string fields --

dc-srv-protocol = "_tcp" / "_udp" / "_sctp" / "_dccp"
                ; underscore-prefixed transport per RFC 2782

dc-srv-service  = dc-underscore-label
                ; underscore-prefixed service name per RFC 6335
```

Internationalized domain names (IDN) are handled in accordance with [\[RFC5891\]](#). Where a parameter description specifies "domain-name", both the ACE ("A-label") and the Unicode ("U-label") forms are accepted.

Service Provider: An entity that offers products and services that are configured or accessed using domain names. These services typically rely on DNS for setup, discovery and/or operation. Examples include web hosting, email services, cloud platforms, and other online applications.

DNS Provider: An entity that offers DNS zone hosting services. DNS Providers are responsible for hosting the DNS zone for a domain name and providing the necessary tools to manage the DNS records. DNS Provider would be an Authoritative server operator for the hosted zones, or would have a contractual relationship with the operator to manage zone distribution over DNS.

User: Refers to the end-user who has means to control domain name's DNS configuration at DNS Provider and wishes to configure it to work with a service provided by a Service Provider.

Service Template/Template: A structured data format that describes a set of configurations for DNS records required by a Service Provider to configure a certain service together with metadata related to the control flow of Domain Connect protocol. A template is used as a means of communication between Service Provider and DNS Provider.

Fully Specified Record Type: A DNS resource record type for which this document, or a specification registered in the IANA "Domain Connect Fully Specified Record Types" registry (see [Section 12.2](#)), defines a breakdown of the record's RDATA into individual, normatively-typed template fields rather than carrying the RDATA opaquely.

Computed Record Type: A template record type that does not correspond to a single DNS resource record placed verbatim into the zone. Instead, it instructs the DNS Provider to derive one or more DNS resource records by a computation defined for that type. The resulting record types and RDATA need not match the Computed Record Type's own name.

Generic Record Type: Any DNS resource record type, identified by its IANA RR TYPE mnemonic or by the "TYPE" form of [\[RFC3597\]](#), whose RDATA is carried opaquely in the "data" field in DNS presentation format.

Certain labels, parameters, endpoints, and registry entries are present in this specification with "Reserved" status. These items appear in the original Domain Connect community specification [\[DC-SPEC\]](#) and are reserved here to ensure back-compatibility with that specification. Reserved items **MUST NOT** be redefined in any manner that is incompatible with their definition in [\[DC-SPEC\]](#). They **MAY** be used or further defined only in specifications or drafts that extend this document, provided those specifications normatively require this document as their base and are compatible with [\[DC-SPEC\]](#).

4. Protocol design

4.1. Templates

Templates are core to Domain Connect, as they fully describe a service owned by a Service Provider and contain all of the information necessary to enable and operate/maintain the service in the form of a set of records.

The individual records in a template MAY be assigned to a group identified by a "groupId". This allows for the application of templates in different stages. For example, an email provider might first set a TXT record to verify the domain, and later set an MX record to configure email delivery. While done separately, both changes are fundamentally part of the same service.

Templates MAY also contain variable portions, as often values of data in DNS change based on the implementation and/or user of the service (e.g. the IP address of a service, a user id, etc.).

The template is defined by the Service Provider and manually onboarded with the DNS Provider. This is an out-of-band process between the Service Provider and the DNS Provider and not covered by this specification.

4.2. Trust Model

DNS Providers are trusted parties by virtue of their existing full access to the DNS zone. DNS Providers enforce user authorization checks before enacting any DNS zone changes, and present proposed changes to the user in a human-readable form.

Service Providers are not assumed to be trusted by default. A malicious actor may attempt to exploit user trust by impersonating a legitimate Service Provider. Trust between DNS Providers and Service Providers is therefore established out-of-band - typically through an onboarding process involving contractual agreements or template acceptance policies - during which the DNS Provider verifies the legitimacy of the Service Provider and its templates.

Templates define and restrict the permitted scope of DNS modifications. By fixing the records that a Service Provider may affect, templates ensure that no changes beyond the approved scope can be applied.

4.3. Protocol Versioning and Extensibility

This specification is versioned at two independent levels: the HTTP interface and the template format.

All HTTP end-points defined in this document carry a "/v2" path segment immediately after the URL prefix obtained from discovery (see [Section 7](#) and [Section 8](#)). The "2" denotes the protocol generation specified by this document.

A future revision of this protocol that is not backwards compatible with this document MUST use a different version path segment. A revision is not backwards compatible if an implementation of this document, when interacting with an implementation of that revision over the same path segment, could fail to interoperate or could misinterpret a request or a response. Reusing the `"/v2"` segment for such a revision MUST NOT be used. A deployment MAY serve more than one version path segment concurrently.

Revisions SHOULD instead strive to remain backwards compatible and be introduced as extensions under the existing `"/v2"` segment. To make this possible, every extensible data structure of this protocol follows the same principles:

- Ignore unknown: a recipient MUST ignore any element it does not recognize, and the mere presence of an unrecognized element MUST NOT cause the message, template, or request to be rejected. This applies to every extensible structure of this protocol: template-definition properties (see [Section 6.1.1](#)), template record fields (see [Section 6.2.1](#)), Settings End-Point properties (see [Section 7.2.1](#)), apply parameters (see [Section 8.3.2.2](#)), the response of the template query end-point (see [Section 8.2](#)), and the responses of the synchronous flow (see [Section 8.3.5](#) and [Section 8.3.6](#)).
- Must understand: where an extension extends a message such that silently ignoring added elements would be unsafe, the sender marks the affected element names in a `"crit"` element, and a recipient that does not understand a listed element MUST refuse to act on the message rather than process it partially. This mechanism is defined for template definitions (see [Section 6.1.1](#)), for individual template records (see [Section 6.2.1](#)), for the Settings End-Point (see [Section 7.2.1](#)), and for apply parameters (see [Section 8.3.2.2](#)).

Consequently, an extension that can safely be ignored requires no version change. An extension that cannot safely be ignored is signaled per-message through `"crit"`. A change that breaks the framing of the protocol itself requires a new version path segment. For the limits of `"crit"` against deployments that predate this specification, see [Section 11.4](#).

The must-understand mechanism is deliberately not defined for the responses of the synchronous flow. The authoritative record of what was applied is the DNS zone itself, which the Service Provider verifies independently (see [Section 8.4](#)). By the time a response reaches the Service Provider the zone change has already occurred, so there is no operation left to refuse. Extensions to those responses therefore rely on the ignore-unknown rule alone.

The `"_domainconnect"` TXT record (see [Section 7](#)) carries no internal structure and therefore offers no place to hold a version indicator. This is intentional in this revision to maintain compatibility with existing deployments of [\[DC-SPEC\]](#). A future revision MAY define a structured content format for this record, at which point a version indicator can be introduced within that structure. Service Providers implementing this document MUST reject record contents that do not conform to the rules in [Section 7](#), which keeps any such future format distinguishable from a plain authority-and-path value.

The template format is versioned independently of the HTTP interface, through the `"templateSpecVersion"` template property (see [Section 6.1.1](#)).

5. Protocol Flows Overview

5.1. General information

To attach a domain name to a service provided by a Service Provider, the user would first enter their domain name.

Instead of relying on examination of the nameservers and mapping these to DNS Providers, DNS Provider discovery is handled through simple records in DNS and an API. The Service Provider queries for a specific record in the zone that returns a REST endpoint to initiate the protocol. When this endpoint is called, a Domain Connect compliant DNS Provider returns information about that domain and how to configure it using Domain Connect.

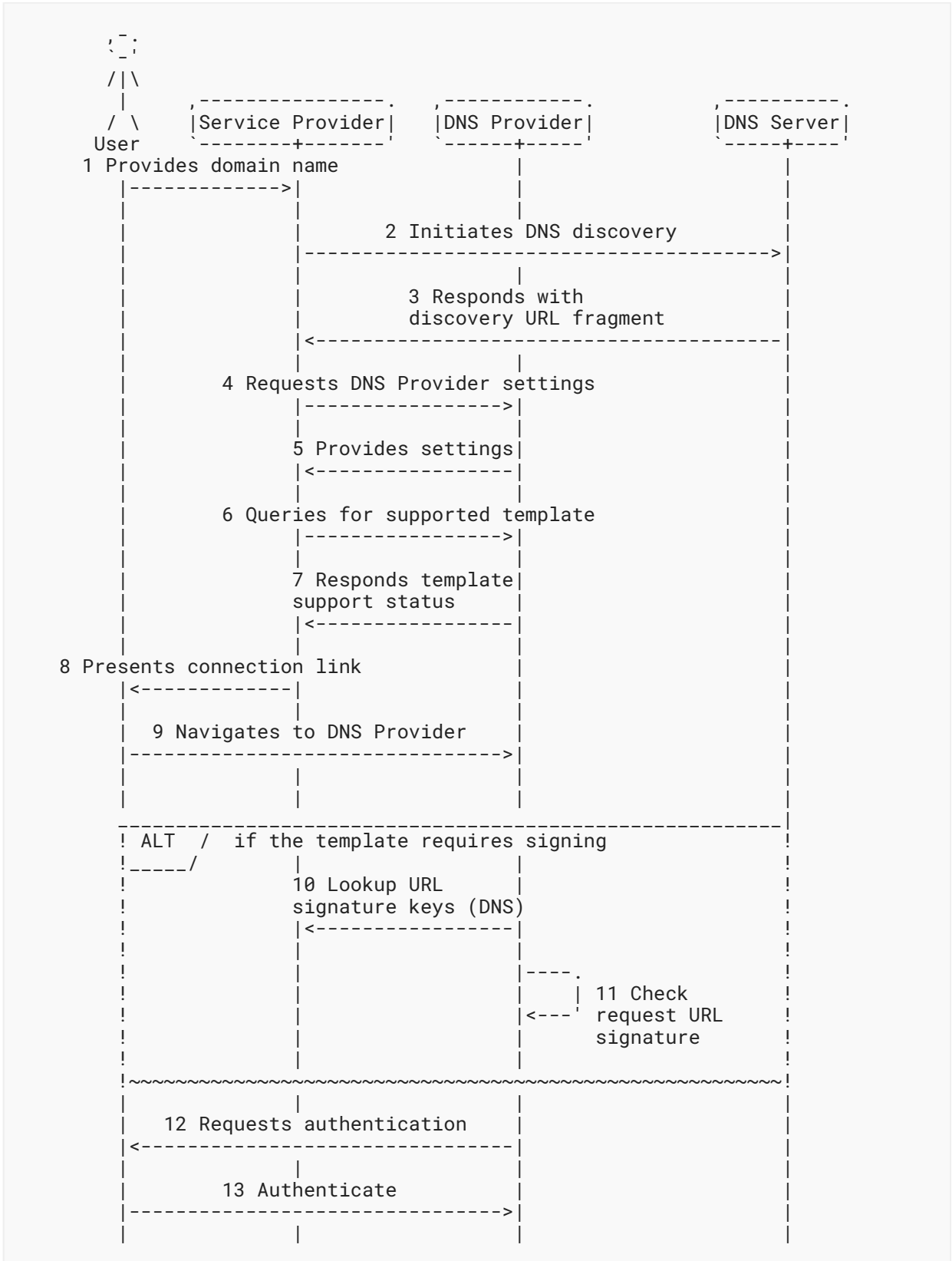
To apply the changes to DNS, this document defines a synchronous web flow. The DNS Provider presents a consent UX to the user and applies the template within the same interaction session.

[[DC-SPEC](#)] also defines an asynchronous flow based on OAuth. Other flows may be defined in the future. Therefore, although it is RECOMMENDED to cover the synchronous flow, implementers MAY decide not to do so and only implement other flows not defined in this document. The signaling offered by DNS Provider discovery allows for that.

5.2. The Synchronous Flow

This flow is tailored for the Service Provider that requires a user-attended, one-time change to DNS configuration. In this flow, the user is present throughout: they authenticate with the DNS Provider and explicitly consent to the changes within a single interaction session.

The term "synchronous" refers to this user-interaction model - not to immediate DNS propagation. The DNS Provider commits to applying the template upon user consent, but the changes MAY be queued internally and propagation to authoritative servers MAY take additional time. Service Providers MUST NOT assume that DNS records are resolvable immediately after the flow completes. See [Section 8.4](#).



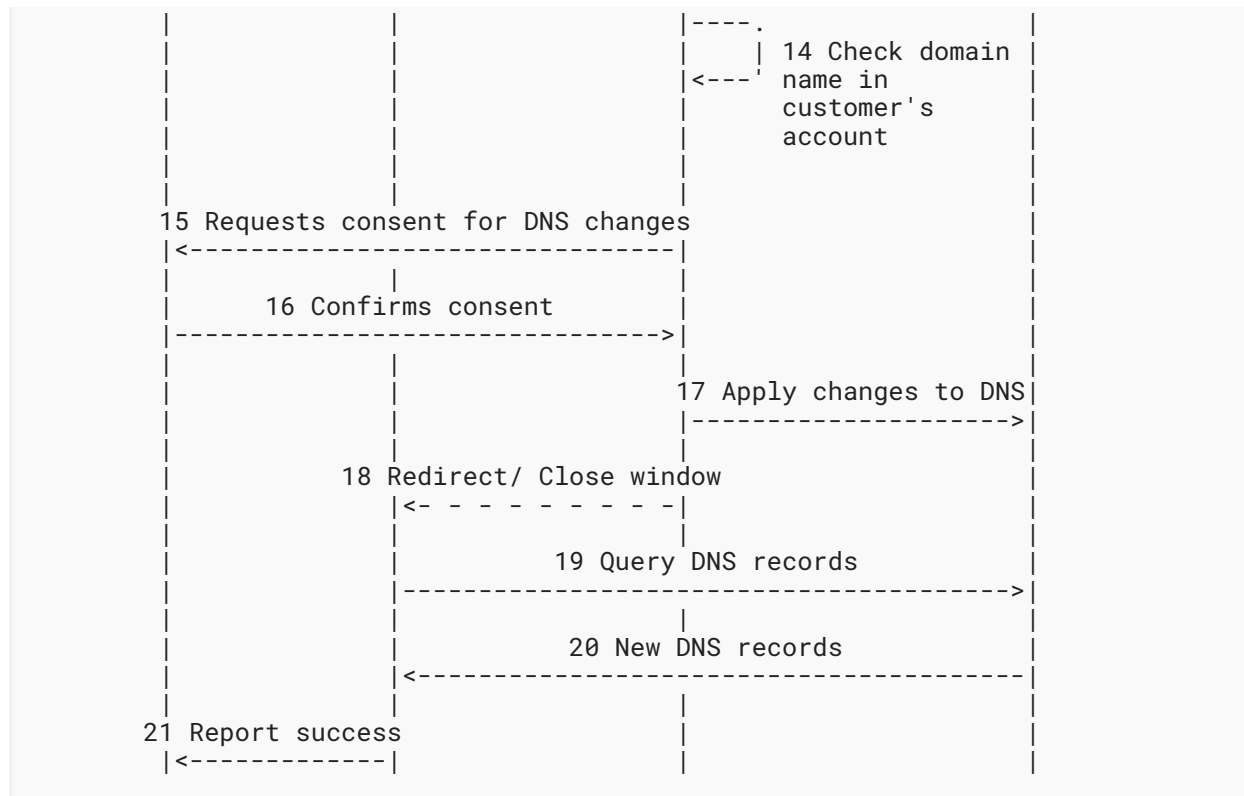


Figure 1: Sequence diagram of Synchronous Flow

Steps:

1. User Provides Domain Name: The user initiates the process by providing their domain name to the Service Provider.
2. Service Provider Initiates DNS Discovery: The Service Provider queries the DNS provider to discover the Domain Connect settings for the given domain.
3. DNS Provider Responds with Discovery URL Fragment: The DNS Provider responds with a URL fragment containing information where to query settings of DNS provider for a domain name.
4. Service Provider Requests DNS Provider Settings: The Service Provider uses the URL fragment to request the complete Domain Connect settings from the DNS Provider.
5. DNS Provider Provides Settings: The DNS Provider provides the settings, including information about API endpoints.
6. Service Provider Queries for Supported Template: The Service Provider checks if the DNS Provider supports the specific template required for the service.
7. DNS Provider Responds with Template Support Status: The DNS Provider confirms if they support the requested template.
8. Service Provider Presents Connection Link: The Service Provider presents a connection link to the user, which leads to the DNS Provider's Domain Connect service.

9. User Navigates to DNS Provider: The user navigates the link and user agent is directed to the DNS Provider's website.
10. DNS Provider Performs URL Lookup and Signature Key Verification (if required): If the template requires signing, the DNS Provider looks up the URL signature keys in DNS.
11. DNS Provider Checks Request URL Signature (if required): The DNS Provider verifies the signature of the request URL.
12. Service Provider Requests Authentication: The Service Provider requests authentication from the user.
13. User Authenticates: The user authenticates with the DNS Provider.
14. DNS Provider Checks Domain Name in Customer's Account: The DNS Provider verifies that the user is authorized to make changes to the domain's DNS zone.
15. DNS Provider Requests Consent for DNS Changes: The DNS Provider asks the user for consent to apply the changes to the DNS zone.
16. User Confirms Consent: The user confirms their consent to the DNS changes.
17. DNS Provider Applies Changes to DNS: The DNS Provider applies the changes to the zone.
18. DNS Provider Redirects or User Flow Termination: The DNS Provider either redirects the user agent back to the Service Provider or gracefully terminates the user flow.
19. Service Provider Queries DNS Records: The Service Provider queries the DNS records to verify that the changes have been applied.
20. DNS Server Returns New DNS Records: The DNS Server returns the updated DNS records.
21. Service Provider Reports Success: The Service Provider reports to the user that the domain has been successfully connected to the service.

6. Domain Connect Objects and Templates

6.1. Template Definition

A template is defined as a standard JSON data structure containing the following data. Field values **MUST** be defined unless otherwise indicated.

The template JSON structure defined in this section is identified by the media type `"application/domainconnect-template+json"` (see [Section 12.8](#)).

Properties of the template definition:

"Service Provider Id":
JSON key: `"providerId"`
Type: `"String"`

(REQUIRED) The unique identifier of the Service Provider that created this template. This is used in the URLs to identify the Service Provider.

The value **MUST** conform to the `dc-id` syntax (see [Section 3](#)).

To ensure non-coordinated uniqueness, this SHOULD be the domain name of the Service Provider (e.g. `exampleservice.example`).

"Service Provider Name":

JSON key: `"providerName"`

Type: `"String"`

(REQUIRED) The name of the Service Provider, suitable for display to the user.

The value MUST conform to the `dc-display-name` syntax (see [Section 3](#)).

"Service Id":

JSON key: `"serviceId"`

Type: `"String"`

(REQUIRED) The name or identifier of the template. This is used in URLs to identify the template.

The value MUST conform to the `dc-id` syntax (see [Section 3](#)).

"Service Name":

JSON key: `"serviceName"`

Type: `"String"`

(REQUIRED) The name of the service, suitable for display to the user.

The value MUST conform to the `dc-display-name` syntax (see [Section 3](#)).

"Version":

JSON key: `"version"`

Type: `"Integer"`

(OPTIONAL) If present this represents a version of the template and SHOULD be changed with each update of the template content. This opaque value is mainly informational to improve communication and transparency between providers.

The value MUST conform to the `dc-version` syntax (see [Section 3](#)).

This is a strict subset of the "JSON number": a positive integer with no leading zeros, no fractional part, and no exponent part.

"Template Specification Version":

JSON key: `"templateSpecVersion"`

Type: "Integer"

(OPTIONAL) The version of the Domain Connect template specification that the template conforms to. This identifies the template format and processing rules defined by this document and its successors.

The value MUST conform to the `dc-version` syntax (see [Section 3](#)). For templates conforming to this document, the value is "1".

"Logo":

JSON key: "logoUrl"

Type: "String"

(OPTIONAL) A graphical logo representing the Service Provider and/or Service for use in any web-based flow. If present this MAY be displayed to the user on the DNS Provider consent UX.

When present, the value MUST be a valid URI [[RFC3986](#)] with scheme "https".

"Description":

JSON key: "description"

Type: "String"

(OPTIONAL) A textual description of what this template does, intended for developer reference. This value is not intended for display to the end user.

The value MUST conform to the `dc-description-text` syntax (see [Section 3](#)).

"Variable Description":

JSON key: "variableDescription"

Type: "String"

(OPTIONAL) A textual description of the template variables, intended for developer reference. This value is not intended for display to the end user.

The value MUST conform to the `dc-description-text` syntax (see [Section 3](#)).

"Synchronous Block":

JSON key: "syncBlock"

Type: "Boolean"

(OPTIONAL) When "true", indicates that this template does not support the synchronous flow. The DNS Provider MUST refuse a synchronous apply request for this template and MUST return an appropriate error to the user.

The default is "false".

"Shared":

JSON key: "shared"

Type: "Boolean"

(OPTIONAL) This flag has been deprecated. It used to indicate that the template allowed a dynamic "providerName" on the query string. It is replaced with the "sharedProviderName" flag in v2.2 of the spec.

"Shared Provider Name":

JSON key: "sharedProviderName"

Type: "Boolean"

(OPTIONAL) When "true", indicates that the caller MAY supply an additional "providerName" parameter at apply time.

The default is "false".

For backward compatibility with DNS Providers prior to v2.2, it is RECOMMENDED that the deprecated "shared" flag also be set.

"Shared Service Name":

JSON key: "sharedServiceName"

Type: "Boolean"

(OPTIONAL) When "true", indicates that the caller MAY supply an additional "serviceName" parameter at apply time.

The default is "false".

"Synchronous Public Key Domain":

JSON key: "syncPubKeyDomain"

Type: "String"

(OPTIONAL) The domain name under which the Service Provider's public signing key TXT record is published. When present, this field signals that digital signing is required for synchronous apply requests.

The value MUST conform to the dc-pubkey-domain syntax (see [Section 3](#)).

"Synchronous Redirect Domains":

JSON key: `"syncRedirectDomain"`

Type: `"String"`

(OPTIONAL) A comma-separated list of domain names to which the DNS Provider is permitted to send the post-apply redirect in the synchronous flow.

The value MUST conform to the `"dc-host-list"` syntax (see [Section 3](#)).

"Multiple Instance":

JSON key: `"multiInstance"`

Type: `"Boolean"`

(OPTIONAL) When `"true"`, indicates that the template is designed to be applied multiple times to the same domain and host.

The default is `"false"`.

"Warn Phishing":

JSON key: `"warnPhishing"`

Type: `"Boolean"`

(DEPRECATED) This flag is deprecated and its code point is reserved (see [Section 3, Paragraph 13](#)). Phishing warnings are now the default behavior in the absence of other security mechanisms (see [Section 11.1](#)). This flag MUST NOT be relied upon and MUST NOT be set in new templates. A DNS Provider MUST NOT treat the absence of this flag as a signal that a template is not susceptible to phishing.

"Host Required":

JSON key: `"hostRequired"`

Type: `"Boolean"`

(OPTIONAL) When `"true"`, indicates that the template is designed to work only when both `"domain"` and `"host"` apply parameters are provided.

The default is `"false"`.

"Critical Properties":

JSON key: `"crit"`

Type: `"Array of String"`

(OPTIONAL) A list of the names of template properties that an implementation is required to understand and process for the template to be applied correctly.

Each entry MUST conform to the `dc-template-property` syntax (see [Section 3](#)).

When present, the `"crit"` property MUST NOT be empty.

"Template Records":

JSON key: `"records"`

Type: `"Array of Template Records"`

(REQUIRED) A list of records for the template.

6.1.1. Template Property Extensibility

The set of template-definition properties defined above MAY be extended. To keep template definitions forward-compatible and to avoid name collisions between this specification, future standards-track extensions, and vendor-specific additions, the property name space is partitioned as follows:

- Property names without any prefix are reserved for this specification and for future standards-track extensions that normatively require this document as their base. Such names are managed through the IANA "Domain Connect Template Properties" registry (see [Section 12.4](#)). Implementations MUST NOT introduce private or vendor-specific properties using unprefixed names.
- Property names beginning with `"vnd."` are reserved for vendor-specific and private use. A vendor SHOULD further namespace such properties with a vendor identifier (e.g. `"vnd.example.fooBar"`), and such names MAY be registered in the IANA "Domain Connect Template Properties" registry (see [Section 12.4](#)). Properties using this prefix MUST NOT be assumed to be interoperable across implementations, and the property name MUST conform to the `dc-vnd-property` syntax (see [Section 3](#)).

Both DNS Providers and Service Providers process template definitions. An implementation of either role MUST ignore any template property it does not recognize. The mere presence of an unrecognized property MUST NOT cause the template to be rejected. This applies equally to unprefixed properties added by later revisions of this specification and to `"vnd."` properties.

The `"crit"` property (see [Section 6.1](#)) overrides this default for the properties it lists: if an implementation does not understand and cannot process one or more of those properties, it MUST reject the template and MUST NOT apply it. A listed name that does not correspond to a property present in the template definition, is by definition understood.

An example of vendor-specific template properties and a template-level `"crit"` property is given in [Appendix A.5](#).

The `"templateSpecVersion"` property (see [Section 6.1](#)) identifies the version of the template specification the template conforms to. An implementation that does not support the indicated template specification version MUST reject the template and MUST NOT apply it. A template that omits `"templateSpecVersion"` is processed as if it declared version `"1"`.

6.2. Template Record

Each template record is an entry that contains a type and several other values depending on the type.

Many of these values can contain variables, which are expressed as strings surrounded with "%" or special variable "@" (See: [Section 9.1](#)). Variables are replaced with values when the template is applied.

Each record MUST contain the following elements unless otherwise specified.

Properties of the template record definition:

"Type":

JSON key: "type"

Type: "enum"

(REQUIRED) Describes the type of record in DNS, or the operation impacting DNS.

Record types fall into three classes (see [Section 3](#)):

Fully Specified Record Types - A, AAAA, CNAME, MX, TXT, SRV, NS - whose RDATA is broken into dedicated template fields. The complete set is managed by the IANA "Domain Connect Fully Specified Record Types" registry (see [Section 12.2](#)).

Computed Record Types - which instruct the DNS Provider to derive DNS records by a defined computation rather than write a verbatim record. "SPFM" is the Computed Record Type defined in this document.

Generic Record Types - any other IANA-registered RR TYPE mnemonic, or the "TYPE" form of [\[RFC3597\]](#), whose RDATA is carried opaquely in the "data" field.

The DNS Provider MUST support the core Fully Specified Record Types A, AAAA, CNAME, MX, TXT, SRV.

The DNS Provider SHOULD support the "SPFM" Computed Record Type for high interoperability with existing templates.

Support for NS, for Generic Record Types, and for any Fully Specified Record Type registered by another specification is OPTIONAL.

The value MUST conform to the dc-record-type syntax (see [Section 3](#)).

"Group ID":

JSON key: "groupId"

Type: "String"

(OPTIONAL) This parameter identifies the group the record belongs to when applying changes.

The value MUST conform to the `dc-id` syntax (see [Section 3](#)) and MUST NOT contain variable expressions.

"Essential":

JSON key: `"essential"`

Type: `"enum"`

(OPTIONAL) This parameter indicates how the record is treated during conflict detection with existing templates.

If the DNS Provider is not implementing applied template state in DNS this is ignored.

Always (default) - record MUST be applied and kept with the template

OnApply - record MUST be applied but can be later removed without dropping the whole template

The value MUST conform to the `dc-essential` syntax (see [Section 3](#)).

If omitted, the value MUST be assumed to be `"Always"`.

"Host":

JSON key: `"host"`

Type: `"String"`

(REQUIRED) The host for A, AAAA, CNAME, NS, TXT, MX and Generic Record Type values.

This value is relative to the applied host and domain, unless trailed by a `"."`.

A value of empty or `"@"` indicates the root of the applied host and domain. In other words `"[host.]example.com."`.

When used in a template definition, the value MUST conform to the `dc-host-tmpl` syntax (see [Section 3](#)).

After variable substitution, the resolved value MUST conform to the `domain-name` syntax (see [Section 3](#)) or be empty.

"Name":

JSON key: `"name"`

Type: `"String"`

The name for the SRV record.

This value is relative to the applied host and domain, unless trailed by a ".".

A value of empty or "@" indicates the root of the applied host and domain.

When used in a template definition, the value MUST conform to the `dc-host-tmpl` syntax (see [Section 3](#)).

After variable substitution, the resolved value MUST conform to the domain-name syntax (see [Section 3](#)) or be empty.

"Points To":

JSON key: `"pointsTo"`

Type: `"String"`

The pointsTo location for A, AAAA, CNAME, NS and MX records.

When used in a template definition, the value MUST conform to the `"dc-pointsto-tmpl"` syntax for `"CNAME"/"MX"` and to the `"dc-pointsto-nat-tmpl"` syntax for A/AAAA/NS (see [Section 3](#)).

For `"CNAME"`, `"MX"` and `"NS"` records the resolved value is a domain name and MUST be treated as fully qualified, whether or not it ends with a ".". It MUST NOT be interpreted relative to the applied host and domain.

After variable substitution, the resolved value MUST conform to the presentation format of the corresponding resource record type.

"TTL":

JSON key: `"ttl"`

Type: `"Int"` or string representation of `"Int"`

The time-to-live for the record in DNS. Valid for A, AAAA, CNAME, NS, TXT, MX, and SRV records. This SHOULD NOT contain variables unless absolutely necessary.

When used in a template definition, the value MUST conform to the `dc-ttl-tmpl` syntax (see [Section 3](#)).

After variable substitution, the resolved value MUST conform to the `dc-ttl-value` syntax.

This value, no matter if variable or constant, is understood as "best effort" by DNS Provider and MAY be limited or adjusted by local policy at runtime or during template onboarding, like applying a certain minimum or maximum value of TTL or an enumeration of TTL values

supported by the DNS Provider. The DNS Provider SHOULD NOT reject template application because of invalid value, rather pick the nearest supported value or a default, in order to avoid necessity of per provider adjustment to the application flow.

Records forming the same RRset (i.e., sharing "host"/"name" and record type) SHOULD be given the same "ttl" value in the template definition, since [\[RFC2181\]](#) Section 5.2 requires all resource records in an RRset to share a single TTL.

Support of variables in this field is OPTIONAL for DNS Provider.

"Data":

JSON key: "data"

Type: "String"

For TXT record "data" contains TXT-DATA [\[RFC1035\]](#) in presentation format. For a single "<character-string>" the heading and trailing " character MAY be omitted even if space character is present in the value.

"data" MUST NOT be present in a template record for any Fully Specified or Computed Record Type, which use type-specific data fields defined in this specification.

For a Generic Record Type, this field contains the canonical presentation format of the record, following [\[RFC3597\]](#) generic or type-specific encoding.

"TXT Conflict Matching Mode":

JSON key: "txtConflictMatchingMode"

Type: "String"

Describes how conflicts on the TXT record are detected. Possible values are None, All, or Prefix. [See below \(Section 10.4\)](#).

The value MUST conform to the dc-txt-conflict-mode syntax (see [Section 3](#)).

If omitted, the value MUST be assumed to be "None".

"TXT Conflict Matching Prefix":

JSON key: "txtConflictMatchingPrefix"

Type: "String"

The prefix to detect conflicts when txtConflict-MatchingMode is "Prefix". [See below \(Section 10.4\)](#).

The value MUST conform to the dc-conflict-prefix syntax (see [Section 3](#)).

"Priority":

JSON key: "priority"

Type: "Int" or string representation of "Int"

The priority for an MX or SRV record.

When used in a template definition, the value MUST conform to the `dc-uint16-tmpl` syntax (see [Section 3](#)).

After variable substitution, the resolved value MUST conform to the `dc-uint16-value` syntax.

Support of variables in this field is OPTIONAL for DNS Provider.

"Weight":

JSON key: "weight"

Type: "Int" or string representation of "Int"

The weight for the SRV record.

When used in a template definition, the value MUST conform to the `dc-uint16-tmpl` syntax (see [Section 3](#)).

After variable substitution, the resolved value MUST conform to the `dc-uint16-value` syntax.

Support of variables in this field is OPTIONAL for DNS Provider.

"Port":

JSON key: "port"

Type: "Int" or string representation of "Int"

The port for the SRV record.

When used in a template definition, the value MUST conform to the `dc-uint16-tmpl` syntax (see [Section 3](#)).

After variable substitution, the resolved value MUST conform to the `dc-uint16-value` syntax.

The resolved value is further constrained to the range 0-65535 as defined by [\[RFC6335\]](#).

Support of variables in this field is OPTIONAL for DNS Provider.

"Protocol":

JSON key: "protocol"

Type: "String"

The protocol for the SRV record.

The value MUST conform to the `dc-srv-protocol` syntax (see [Section 3](#)).

"Service":

JSON key: `"service"`

Type: `"String"`

The symbolic service name for the SRV record.

The value MUST conform to the `dc-srv-service` syntax (see [Section 3](#)).

"Target":

JSON key: `"target"`

Type: `"String"`

The target hostname for the SRV record as defined in Section 3.1 of [\[RFC2782\]](#).

When used in a template definition, the value MUST conform to the `dc-pointsto-tmpl` syntax (see [Section 3](#)), or be the single label `"."` to indicate that the service is not available.

The resolved value MUST be treated as fully qualified, whether or not it ends with a `"."`. It MUST NOT be interpreted relative to the applied host and domain.

After variable substitution, the resolved value MUST conform to the `domain-name` syntax (see [Section 3](#)), or be `"."`.

"SPF Rules":

JSON key: `"spfRules"`

Type: `"String"`

These are desired rules for the SPF TXT record. These rules SHOULD be merged with other SPF records into final SPF TXT record. See [Section 9.4](#). The value MUST contain only mechanism and modifier terms as defined in [\[RFC7208\]](#) Section 5, excluding the version prefix (`"v=spf1"`) and the terminating `"all"` qualifier. The DNS Provider MUST validate the syntax before merging.

"Critical Fields":

JSON key: `"crit"`

Type: `"Array of String"`

(OPTIONAL) A list of the names of fields of this record that an implementation is required to understand and process for the record to be applied correctly.

Each entry MUST conform to the `dc-record-field` syntax (see [Section 3](#)).

When present, the `"crit"` field MUST NOT be empty and MUST NOT contain variables.

The following table lists the fields that are defined for each record type by this document. Fields beyond those listed are governed by [Section 6.2.1](#).

Type	Required fields
"A"	"host", "pointsTo", "ttl"
"AAAA"	"host", "pointsTo", "ttl"
"CNAME"	"host", "pointsTo", "ttl"
"NS"	"host", "pointsTo", "ttl"
"TXT"	"host", "data", "ttl", "txtConflictMatchingMode", "txtConflictMatchingPrefix"
"MX"	"host", "pointsTo", "ttl", "priority"
"SRV"	"name", "target", "ttl", "priority", "protocol", "service", "weight", "port"
"SPFM"	"host", "spfRules"
Generic types	"host", "data", "ttl"

Table 1: Fields per record type

6.2.1. Template Record Extensibility

The set of template record fields defined above MAY be extended. Record field name space is partitioned as follows:

- unprefixed names are reserved for this specification and for future standards-track extensions that normatively require this document as their base
- names beginning with `"vnd."` are reserved for vendor-specific and private use.

Every record field name MUST conform to the `dc-record-field` syntax (see [Section 3](#)).

Record fields fall into two categories. Protocol-level fields - `"type"`, `"groupId"`, `"essential"` and `"crit"` - are defined by this protocol and are applicable to a record irrespective of its `"type"`. Because such a field must carry the same meaning for every record type, unprefixed protocol-level field names are coordinated through the IANA "Domain Connect Protocol-Level Record Fields" registry (see [Section 12.5](#)). All remaining fields are type-specific: they carry the RDATA of a particular record type or are otherwise meaningful only for certain types.

The two categories MUST NOT collide. A protocol-level field name, being valid for every record type, MUST NOT reuse a name already used as a type-specific field by any record type, and a specification defining type-specific fields MUST NOT reuse a name registered as protocol-level (see [Section 12.5](#)).

An implementation MUST ignore any record field it does not recognize, and MUST ignore any recognized field that is not defined for the record's type. The mere presence of such a field MUST NOT cause the record or the template to be rejected.

The "crit" record field (see [Section 6.2](#)) overrides this default for the fields it lists: if an implementation does not understand and cannot process one or more of those fields, it MUST reject the template and MUST NOT apply it. Rejection applies to the template as a whole, not only to the record carrying the "crit" field, because a partially applied template would leave the service configured incorrectly. A listed name that does not correspond to a field present in the record is by definition understood.

An example of vendor-specific record fields and a record-level "crit" field is given in [Appendix A.5](#).

6.3. Template Considerations

6.3.1. Template Scope

Templates MUST be considered as scoped to the "domain" and "host" apply parameter pair. The "host" value is specified in the apply URL.

The template's "host" or "name" field values are interpreted relative to this scope, as described in [Section 9.3](#).

6.3.2. Sub Domains

The recommended way to configure records on a Sub Domain is to use the "host" apply parameter rather than embedding sub-domain labels in template "host" field values as variables. This keeps the template scope unambiguous and enables correct conflict detection and template state tracking. Template "host" field values containing variables that resolve to sub-domain labels cause the template scope to be indeterminate at consent time, which prevents accurate conflict detection.

To add a record at the Zone Apex even when a Sub Domain is specified in the apply request, the template "host" field value "%domain%" MAY be used, which resolves to the absolute Zone Apex name regardless of the "host" parameter.

When a template is not applicable at the Zone Apex - for example because it contains a CNAME record or any other record type that is incompatible with Apex placement - the "hostRequired" flag SHOULD be set to "true" in the template definition (see [hostRequired \(Section 6.1\)](#)).

6.3.3. Variable Scope Minimization

It is noted that as a best practice the variable portions SHOULD be constrained to as small as possible a portion of the resulting DNS record.

For example, say a Service Provider requires a CNAME of one of three values for their users: "s01.example.com", "s02.example.com", and "s03.example.com".

The value in the template could simply contain "%servercluster%", and the fully qualified string passed in. Alternatively, the value in the template could contain "s%var%.example.com" and a value of "01", "02", or "03" passed in. By placing more fixed data into the template, the template is less prone to error or misuse and allows better review of intent by the DNS Provider when onboarding the template.

6.4. Public Key Publication

The Service Provider MUST publish their public signing key in one or more DNS TXT records at the host formed by prepending the "key" apply parameter value as a label to the "syncPubKeyDomain" template field value.

Each TXT record MUST conform to the following grammar:

```

dc-pubkey-record = dc-pubkey-kv *( "," dc-pubkey-kv )
dc-pubkey-kv     = ( "p" "=" dc-frag-index )
                  / ( "d" "=" dc-frag-data )
                  / ( "a" "=" dc-alg-id )
                  / ( "t" "=" dc-key-type )

dc-frag-index    = 1*DIGIT
                  ; positive decimal integer;
                  ; used for p=
dc-frag-data     = 1*( ALPHA / DIGIT / "+" / "/" / "=" )
                  ; standard base64 per RFC 4648;
                  ; used for d=
dc-alg-id        = 1*( ALPHA / DIGIT / "-" )
                  ; JWS algorithm identifier per RFC 7518;
                  ; used for a=
dc-key-type      = 1*( ALPHA / DIGIT / "-" )
                  ; public key format identifier;
                  ; used for t=

```

Properties of the public key TXT record:

"Fragment Index":

Key: "p"

(REQUIRED) The sequence number of this key fragment.

The value MUST conform to the dc-frag-index rule above: a positive decimal integer.

Fragments MUST be reassembled in ascending numeric order of "p".

"Fragment Payload":

Key: "d"

(REQUIRED) A Fragment of base64-encoded key material.

The value MUST conform to the `dc-frag-data` rule above: standard base64 encoding per [\[RFC4648\]](#).

"Signing Algorithm":

Key: "a"

(OPTIONAL) The JWS algorithm identifier for the signing algorithm.

The value MUST conform to the `dc-alg-id` rule above and MUST be registered in the IANA "JSON Web Signature and Encryption Algorithms" registry established by [\[RFC7518\]](#).

If omitted, MUST be assumed to be "RS256".

Support for "RS256" is MANDATORY for both DNS Providers and Service Providers.

"Public Key Format":

Key: "t"

(OPTIONAL) The format of the public key.

The value MUST conform to the `dc-key-type` rule above.

If omitted, MUST be assumed to be "x509".

A Service Provider MUST publish exactly one public key on any given host name. To use multiple keys, e.g. for key rotation, the Service Provider MUST publish each key on a different host name within `"syncPubKeyDomain"`. The `"key"` apply parameter identifies which host to query for each request and MUST be included in the signed apply request.

To account for DNS record size limits, a public key MAY be split across multiple TXT records at the same host. All fragments MUST be reassembled in ascending numeric order of `"p"` before use. All such TXT records at the same host therefore belong to the same key: the values `"a"` and `"t"` MUST be equal across them, and their `"d"` fragments MUST have distinct `"p"` indices.

Given the public key (line breaks for brevity):

```
MIIBIjANBgqhkiG9w0BAQEFAAOCQA8AMIIBCgKCAQEA18SgvpmeasN4BHkkv0SBjAzIc
4grYLjiAXRtNiBUiGUDMeTzQrKtSWvy9NuxU1dIHCZy9o1CrKNG5EzLIZLNyMfi6qiXnM
+HMD4byp97zs/3D39Q8iR5poubQcRaGozWx8yQpG00cVdmEVcTfyR/XSEWC5u16EBNvRn
NAOAvZYUdWqVyQvXsjnxQot8KcK0QP8iHpoL/1dbdRy2opRPQ2FdZpovUgknybq/6FkeD
tW7uCQ6Mvu4QxcUa3+WP9nYHKtgWip/eFxpEb+qLvcLHf1h0JXtxLVdyy60Lk3f2JRYUX
2ZZVDvG3biTpeJz6iRzjGg6MfGxXZHjI8weDjXrJwIDAQAB
```

Published at `"_dcpubkeyv1.<syncPubKeyDomain>"` as (line breaks for brevity):

EXAMPLE: Example: public key published as DNS TXT records

```
p=1,a=RS256,d=MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA18SgvpmeasN
4BHkKv0SBjAzIc4grYLjiAXRtNiBUiGUDMeTzQrKTsWvy9NuxU1dIHCZy9o1CrKNG5EzL
IZLNyMfI6qiXnM+HMD4byp97zs/3D39Q8iR5poubQcRaGozWx8yQpG00cVdmEVcTfy

p=2,a=RS256,d=R/XSEWC5u16EBNvRnNA0AvZYUdWqVyQvXsjnxQot8KcK0QP8iHpoL/1
dbdRy2opRPQ2FdZpovUgknybq/6FkeDtW7uCQ6Mvu4QxcUa3+WP9nYHKtgWip/eFxpeb+
qLvclHf1h0JXtxLVdyy60Lk3f2JRYUX2ZZVDvG3biTpeJz6iRzjGg6MfGxXZHjI8

p=3,a=RS256,d=weDjXrJwIDAQAB
```

7. DNS Provider Discovery

7.1. "_domainconnect" Resource Record

To facilitate discovery of the DNS Provider from a domain name DNS is utilized. This is done by returning a TXT record for "_domainconnect" in the zone.

The record content represents an authority and path part of the settings REST API URL.

EXAMPLE 1: An example of the contents of this record:

```
domainconnect.virtucondomains.example
```

EXAMPLE 2: An example of the contents of this record including a path segment:

```
domainconnect.virtucondomains.example/dc
```

"_domainconnect" TXT record content, when prepended with `https://` schema and appended with `/v2` path segment, MUST form a valid URL [RFC3986]. "_domainconnect" TXT record MUST contain the authority part of the URL and MAY contain a path part. "_domainconnect" MUST not contain schema, query or fragment part of an URL.

The DNS query for the "_domainconnect" TXT record MAY return more than one TXT record. The Service Provider MUST reject any returned record whose content does not satisfy the validity rules above. If, after this filtering, at least one valid record remains, the Service Provider MUST process at least one of them. The Service Provider MAY iterate over the remaining valid records, attempting discovery with each in turn, until a valid Settings End-Point response is obtained.

7.1.1. Deployment considerations

As a practical matter of implementation, the DNS Provider may or may not contain a copy of this data in each and every zone. Instead, the DNS Provider MUST simply respond to the DNS query for the "_domainconnect" TXT record with the appropriate data.

How this is implemented is up to the DNS Provider.

For example, the DNS Provider may not store the data inside a TXT record for the domain, opting instead to put a CNAME in the zone and have the TXT record in the target of the CNAME. Another DNS Provider may simply respond with the appropriate records at the DNS layer without having the data in each zone.

7.2. Settings End-Point

The URL prefix returned in "`_domainconnect`" Resource Record MUST be subsequently used by the Service Provider to determine the additional settings for using Domain Connect on this domain at the DNS Provider. This is done by calling a REST API.

Normative URI template of the Settings End-Point per [\[RFC6570\]](#):

```
GET
```

```
https://{+_domainconnect}/v2/{domain}/settings
```

"`_domainconnect`" parameter is the URL prefix returned in the "`_domainconnect`" TXT record.

This MUST return a JSON structure containing the settings to use for Domain Connect on the domain name (passed in on the path) at the DNS Provider. This JSON structure MUST contain the following fields unless otherwise specified.

Properties of the settings data structure:

"Provider Id":

JSON key: "`providerId`"

Type: "`String`"

(REQUIRED) Unique identifier for the DNS Provider.

The value MUST conform to the `dc-id` syntax (see [Section 3](#)).

To allow for non-coordinated uniqueness, this SHOULD be the domain name of the DNS Provider (e.g. `virtucondomains.example`).

"Provider Name":

JSON key: "`providerName`"

Type: "`String`"

(REQUIRED) The name of the DNS Provider.

The value MUST conform to the `dc-display-name` syntax (see [Section 3](#)).

"Provider Display Name":

JSON key: "providerDisplayName"

Type: "String"

(OPTIONAL) The name of the DNS Provider that SHOULD be displayed by the Service Provider.

This MAY change per domain for some DNS Providers that power multiple brands.

When present, the value MUST conform to the dc-display-name syntax (see [Section 3](#)).

"UX URL Prefix for Synchronous Flows":

JSON key: "urlSyncUX"

Type: "String"

(OPTIONAL) The URL Prefix for linking to the UX of Domain Connect for the synchronous flow at the DNS Provider. If not returned, the DNS Provider is not supporting the synchronous flow on this domain.

This MUST be a valid URI [[RFC3986](#)] with scheme "https", MUST include an authority component, and MUST NOT contain a query component or fragment component.

The URI MAY include a path component.

"API URL Prefix":

JSON key: "urlAPI"

Type: "String"

(REQUIRED) The URL Prefix for the REST API.

This MUST be a valid URI [[RFC3986](#)] with scheme "https", MUST include an authority component, and MUST NOT contain a query component or fragment component.

The URI MAY include a path component.

"Name Servers":

JSON key: "nameServers"

Type: "List of String"

(OPTIONAL) The list of nameserver hostnames for the zone held by this DNS Provider (typically derived from the zone's NS records).

Each entry MUST conform to the "domain-name" syntax (see [Section 3](#)).

"Critical Properties":

JSON key: "crit"

Type: "List of String"

(OPTIONAL) A list of the names of settings properties that a Service Provider is required to understand and process in order to interact correctly with this DNS Provider.

Each entry MUST conform to the dc-settings-property syntax (see [Section 3](#)).

When present, the "crit" property MUST NOT be empty.

"Reserved":

JSON key: "urlAsyncUX"

(Reserved. See [Section 3, Paragraph 13](#).)

"Reserved":

JSON key: "width"

(Reserved. See [Section 3, Paragraph 13](#).)

"Reserved":

JSON key: "height"

(Reserved. See [Section 3, Paragraph 13](#).)

"Reserved":

JSON key: "urlControlPanel"

(Reserved. See [Section 3, Paragraph 13](#).)

EXAMPLE: Example Settings End-Point response

```
{
  "providerId": "virtucondomains.example",
  "providerName": "Virtucon Domains",
  "providerDisplayName": "Virtucon Domains",
  "urlSyncUX": "https://domainconnect.virtucondomains.example/sync",
  "urlAPI": "https://api.domainconnect.virtucondomains.example",
  "urlControlPanel": "https://domaincontrolpanel.virtucondomains.example/?domain=%domain%",
  "nameServers": ["ns01.virtucondomainsdns.example", "ns02.virtucondomainsdns.example"]
}
```

When constructing a deep link using "urlControlPanel", the Service Provider MUST replace any occurrence of the literal token "%domain%" with the percent-encoded ACE-form domain name per [RFC3986]. The "%domain%" token MUST be the only substitution token used.

The Service Provider MAY compare the "nameServers" list against the authoritative nameservers obtained from the DNS registry or an authoritative NS query, to verify that the correct DNS Provider has been reached. A mismatch may indicate a stale or incorrect "_domainconnect" TXT record, or a deliberate administrative change such as pre-provisioning with a new DNS provider.

Discovery MUST work on the Zone Apex only. Bear in mind that zones can be delegated to other users, making this information valuable to Service Providers since DNS changes may be different for a Zone Apex vs. a Sub Domain for an individual service.

The Service Provider MUST handle the condition when a query for the "_domainconnect" TXT record succeeds, but a call to query for the JSON fails. This can happen if the zone is hosted with another DNS Provider, but contains an incorrect "_domainconnect" TXT record.

The DNS Provider MUST return a 404 HTTP error code if they do not contain the zone.

Status	Response	Description
Success	2xx	A response of an http status code of 2xx indicates that the call was successful. The response is the JSON described above.
Not Found	404	A response of a 404 indicates that the DNS Provider does not have the zone.

Table 2: HTTP status codes for the settings end-point

7.2.1. Settings Property Extensibility

Clients MUST ignore any JSON properties in the settings response that are not defined in this specification or not recognized from an applicable extension. DNS Providers MAY include additional JSON properties in the settings response beyond those defined in this specification, however it is RECOMMENDED that those properties are registered in the "Domain Connect Settings Properties" registry defined in [Section 12.3](#) in order to avoid name conflicts.

Settings property names schema is partitioned as follows:

- unprefixed names are reserved for this specification and for future standards-track extensions that normatively require this document as their base,
- names beginning with "vnd." are reserved for vendor-specific and private use.

Every settings property name MUST conform to the dc-settings-property syntax (see [Section 3](#)).

The "crit" property overrides the ignore-unknown default for the properties it lists: if the Service Provider does not understand and cannot process one or more of those properties, it MUST NOT proceed with the flow against this DNS Provider. A listed name that does not correspond to a property present in the settings response is by definition understood.

EXAMPLE: Example Settings End-Point response with vendor-specific properties

```
{
  "providerId": "virtucondomains.example",
  "providerName": "Virtucon Domains",
  "providerDisplayName": "Virtucon Domains",
  "urlSyncUX": "https://domainconnect.virtucondomains.example/sync",
  "urlAPI": "https://api.domainconnect.virtucondomains.example",
  "vnd.example.contact": {
    "supportUrl": "https://support.virtucondomains.example/",
    "email": "support@virtucondomains.example"
  },
  "vnd.example.euSyncUX": "https://eu.virtucondomains.example",
  "crit": ["vnd.example.euSyncUX"]
}
```

8. Applying Domain Connect

8.1. Endpoints

The Domain Connect endpoints returned in the JSON during discovery are in the form of URLs.

The first set of endpoints are for the UX that the Service Provider links to. These are for the synchronous flow where the user can click to grant consent and have changes applied.

The second set of endpoints are for the REST API.

All endpoints begin with a root URL for the DNS Provider such as:

```
https://connect.dnsprovider.example
```

They MAY also include any path segment at the discretion of the DNS Provider. For example:

```
https://connect.dnsprovider.example/api
```

The root URLs for the UX endpoints and the API endpoints are returned in the JSON payload during DNS Provider discovery.

8.2. Query Supported Template

Normative URI template of the template query end-point per [\[RFC6570\]](#):

```
GET
```

```
{+urlAPI}/v2/domainTemplates/providers/{providerId}/services  
/{serviceId}
```

This URL is be used by the Service Provider to determine if the DNS Provider supports a specific template.

The following table describes the parameters of the URI template:

URI template parameters for the query supported template end-point:

"URL API":

Key: "urlAPI"

(REQUIRED) The base URL of the DNS Provider API, taken from the "urlAPI" field of the settings endpoint response (see [Section 7](#)).

This MUST be a valid URI [[RFC3986](#)] with scheme "https"

"Service Provider Id":

Key: "providerId"

(REQUIRED) Identifier of the Service Provider of the template.

The value MUST conform to the dc-id syntax (see [Section 3](#)).

"Service Id":

Key: "serviceId"

(REQUIRED) Identifier of the template within the Service Provider's namespace.

The value MUST conform to the dc-id syntax (see [Section 3](#)).

Returning a status of 200 without a body indicates the template is supported. The DNS Provider MAY disclose the version of the template in a JSON object with field "version" (see: [version field \(Section 6.1\)](#)) or the full JSON object of deployed template.

The Service Provider MUST ignore any JSON property in this response that it does not recognize. The presence of such a property MUST NOT cause the response to be treated as an error. Where the DNS Provider returns the full template object, the rules of [Template Property Extensibility](#) and [Template Record Extensibility](#) apply to it.

The Service Provider MAY use HTTP content negotiation (see [[RFC9110](#)]) to request the full template object, for example by sending an "Accept" request header of "application/domainconnect-template+json". When the DNS Provider returns the full template object in the

response body, it MUST set the "Content-Type" response header to "application/domainconnect-template+json" (see [Section 12.8](#)). A DNS Provider that does not return the full template object responds as described above.

Returning a status of 404 indicates the template is not supported.

Status	Response	Description
Success	2xx	A response of an http status code of 2xx indicates that the call was successful. The response OPTIONALLY contains the version or template.
Not Found	404	A response of a 404 indicates that the template is not supported

Table 3: https status codes for the Query Supported Template end-point

8.3. Synchronous Flow

8.3.1. Apply Template URL

Normative URI template of the synchronous template apply end-point per [\[RFC6570\]](#):

```
GET

{+urlSyncUX}/v2/domainTemplates/providers/{providerId}/services
/{serviceId}/apply{?domain,host,groupId,providerName,
serviceName,instanceId,redirect_uri,state,crit,properties*,
extensions*}{&sig,key}
```

This is the URL, where the user agent is directed to apply a template to a dns zone the user controls. It is redirected to or linked from the Service Provider to start the synchronous Domain Connect Protocol.

8.3.2. Parameters/properties

For "properties" and "extensions" name/value pairs, parameter names and values MUST be URL-decoded (percent-decoded per [\[RFC3986\]](#)) before processing.

URI template parameters of the apply call in the sync flow:

"URL Sync UX":

Request Parameter: "urlSyncUX"

(REQUIRED) The base URL of the DNS Provider synchronous UX endpoint, taken from the "urlSyncUX" field of the settings endpoint response (see [Section 7](#)).

This MUST be a valid URI [\[RFC3986\]](#) with scheme "https"

"Service Provider Id":Request Parameter: `"providerId"`

(REQUIRED) Identifier of the Service Provider of the template to be applied.

The value MUST conform to the `dc-id` syntax (see [Section 3](#)).**"Service Id":**Request Parameter: `"serviceId"`

(REQUIRED) Identifier of the template to be applied.

The value MUST conform to the `dc-id` syntax (see [Section 3](#)).**"Domain":**Request Parameter: `"domain"`

(REQUIRED) The domain name being configured. This is the Zone Apex (the registered domain or delegated zone).

The value MUST conform to the `domain-name` syntax (see [Section 3](#)).**"Host":**Request Parameter: `"host"`(OPTIONAL) The host name of the Sub Domain within the zone identified by `"domain"`.When present, the value MUST be a single name conforming to `domain-name` (see [Section 3](#)) or an empty string.**"Redirect URI":**Request Parameter: `"redirect_uri"`

(OPTIONAL) The location to direct the user agent to upon successful authorization or upon error.

The value MUST be an absolute URI conforming to [\[RFC3986\]](#).**"State":**Request Parameter: `"state"`

(OPTIONAL) A random and unique string passed along to prevent CSRF, or to pass back state.

The value MUST conform to the `"state"` syntax (see [Section 3](#)).**"Name/Value Pairs":**

Request Parameter: `"properties"`

(REQUIRED) Variable values to be substituted into the template. Each parameter name MUST correspond to a variable name defined in the template and MUST conform to the `variable-name` syntax (see [Section 3](#)).

Each parameter value MUST conform to the `dc-prop-value` syntax (see [Section 3](#)), using the DNS presentation format [[RFC9499](#)].

The parameter value corresponds to the value that will be used when applying the template.

"Extension Parameters":

Request Parameter: `"extensions"`

(OPTIONAL) Name/value pairs of extension parameters as specified in [Section 8.3.2.2](#).

Each parameter name MUST conform to the `dc-dot-param` or `dc-vnd-param` syntax (see [Section 3](#)).

"Provider Name":

Request Parameter: `"providerName"`

(OPTIONAL) Additional display text for the template `"providerName"`, provided by the caller. If `"sharedProviderName"` is not set in the template, this parameter MUST NOT be set. Note: this used to be controlled by the `"shared"` attribute in the template, which has been deprecated.

The value MUST conform to the `dc-display-name` syntax (see [Section 3](#)).

"Service Name":

Request Parameter: `"serviceName"`

(OPTIONAL) Additional display text for the template `"serviceName"`, provided by the caller. If `"sharedServiceName"` is not set in the template, this parameter MUST NOT be set.

The value MUST conform to the `dc-display-name` syntax (see [Section 3](#)).

"Group ID":

Request Parameter: `"groupId"`

(OPTIONAL) Specifies the subset of groups from the template to apply.

The value MUST conform to the `dc-id-list` syntax (see [Section 3](#)).

"Signature":

Request Parameter: `"sig"`

(OPTIONAL) A digital signature of the canonical query string.

The value MUST conform to the `dc-sig` syntax (see [Section 3](#)):

a standard base64-encoded [[RFC4648](#)] signature, URL-encoded when carried in the query string.

See the [Section 8.3.2.1](#) below for the signing procedure.

"Key":

Request Parameter: `"key"`

(OPTIONAL) The DNS host label within the `syncPubKeyDomain` at which the public key TXT record is published.

The value MUST conform to the `dc-key-label` syntax (see [Section 3](#)): a single DNS label, either a plain ACE-form label or an RFC 8552 underscore-prefixed label.

See the [Section 8.3.2.1](#) below.

"Critical Parameters":

Request Parameter: `"crit"`

(OPTIONAL) A comma-separated list of the names of apply parameters that the DNS Provider is required to understand and process for the request to be handled correctly.

The value MUST conform to the `dc-crit-list` syntax (see [Section 3](#)).

See [Section 8.3.2.2](#) below.

Where an apply parameter carries text intended for display to the user, the Service Provider MAY set its value to text localized to match the user agent's language preference (e.g. the HTTP "Accept-Language" header).

An example query string:

```
GET
```

```
https://web-connect.dnsprovider.example/v2/domainTemplates/providers/  
exampleservice.example/services/template1/apply?domain=example.com&  
IP=192.168.42.42&RANDOMTEXT=shm%3A1542108821%3AHello
```

This call indicates that the Service Provider wishes to connect the domain `example.com` to the service using the template identified by the composite key of the provider (`exampleservice.example`) and the service template owned by them (`template1`). In this example, there are two variables in this template, "IP" and "RANDOMTEXT". These variables are passed as name/value pairs.

8.3.2.1. Signing Procedure

Signing the query string is OPTIONAL for templates that do not specify "syncPubKeyDomain". When "syncPubKeyDomain" is present in the template, the Service Provider MUST sign the request and the DNS Provider MUST reject any unsigned apply request (see [Section 8.3.4](#)).

The Service Provider MUST generate the digital signature as follows:

1. Construct the canonical input string:
 - a. Take all apply parameters except "sig" and "key".
 - b. URL-encode each parameter name and value per [\[RFC3986\]](#).
 - c. Sort the parameters in ascending lexicographic order of the URL-encoded parameter name.
 - d. Concatenate them as name=value pairs joined by "&".
2. Sign the canonical input string using the private key corresponding to the public key published at the host identified by the "key" parameter within "syncPubKeyDomain" (see [Section 6.4](#)), using the algorithm identified by the "a" field of the key record (default: "RS256"). The "RS256" identifier denotes RSASSA-PKCS1-v1_5 using SHA-256, as defined in [\[RFC7518\]](#) Section 3.3.
3. The resulting signature MUST be Base64-encoded using the standard alphabet per [\[RFC4648\]](#) Section 4 (conforming to the dc-sig rule).
4. Append the URL-encoded "sig" value and the "key" identifier to the canonical input string to form a signed query string.

The signed query string MUST be used as-is as the query string of the request URL, without re-encoding, adding or re-ordering of query parameters.

EXAMPLE: Example: signed query string parameters

Example canonical input:

```
a=1&b=2&domain=example.net&ip=10.10.10.10&text=a%2Bb
```

Example signature:

```
sig=V2te9zWMU7G3plxBTsmYSJTvn2vzMvNwAjWQ%2BwTe91DxuJhdVf4cVc4vZBYfEYV  
7u5d7PzT07se70rkhyiB7TpoJJW1yB5qHR7HKM5SZldUsdtg5%2B1SzEtIX0Uq8b2mCmQ  
F%2FuJGXpqCyFrEajvpTM7fFKPk1kuctmtkjV7%2BATcvNPLWY7KyE4%2Bqc8jpfN61cP  
5l8iA4krAa3%2BfTro5cmWR8YUJ5yrnRs6KT4b5D71HFv0Uk0sGEUddUUlsyRQKRHUFN6  
HjEya50YDHfZJlYHkHlK0xX6Yqei9QZ2I35U9eJbSvZGQko5beqviWFXdsVDbvd3DYcb  
SHgJq9%2FXoMTTw%3D%3D&key=_dcpubkeyv1
```

8.3.2.2. Apply Parameter Extensibility

The set of apply parameters MAY be extended. To avoid name collisions between this specification, future extensions, and vendor-specific parameters, apply parameter names are governed by the following reservation rules:

- Parameter names beginning with "dc." are reserved for parameters defined by future standards-track extensions that normatively require this document as their base. Implementations MUST NOT use the "dc." prefix for private parameters.
- Parameter names beginning with "vnd." are reserved for vendor-specific and private-use parameters. A vendor SHOULD further namespace such parameters with a vendor identifier (e.g. "vnd.example.foo"). Parameters using this prefix MUST NOT be assumed to be interoperable across DNS Providers.
- Parameter names without any dot-prefix are reserved for this specification and for specifications extending this document that carry over functionality of [DC-SPEC] (see [Section 3, Paragraph 13](#)), such as the client-request query parameters of the OAuth-based asynchronous flow. Implementations MUST NOT define private or vendor-specific parameters using unprefixed names.

The name of every apply parameter MUST conform to the dc-apply-param syntax (see [Section 3](#)), which partitions the parameter name space into the three reservation classes above. Extension parameter names using a reserved prefix accordingly conform to the dc-dot-param (for "dc.") or dc-vnd-param (for "vnd.") rule. A DNS Provider MUST ignore any apply parameter it does not recognize. An unrecognized extension parameter MUST NOT cause the apply request to fail. Extension parameters, when present, are included in the canonical input string for signing exactly as any other parameter (see [Section 8.3.2.1](#)).

The "crit" parameter overrides this default for the parameters it lists. If the DNS Provider does not understand and cannot process one or more of the listed parameters, it MUST NOT apply the template, and MUST signal an error as described in [Section 8.3.6](#) using the error code "invalid_request". A listed name that does not correspond to a parameter present in the request is by definition understood.

EXAMPLE: Example: apply request with a critical vendor-specific parameter

GET

```
https://web-connect.dnsprovider.example/v2/domainTemplates/providers/  
exampleservice.example/services/template1/apply?domain=example.com&  
IP=192.168.42.42&vnd.example.dryRun=1&crit=vnd.example.dryRun
```

8.3.3. Template Apply Request

The Service Provider initiates the synchronous flow by directing the user agent to the Apply Template URL (see [Section 8.3.1](#)), constructed as specified in [Section 8.3.2](#).

The DNS Provider validates the request to ensure that all required parameters are present and valid. This includes also verification of request signature (see [Section 8.3.4](#) below).

If the request is valid, the DNS Provider authenticates the user, validates user's authorization to modify the DNS zone, obtains authorization to apply the template to the DNS and finally applies the changes. This process is described in [Section 10](#).

8.3.4. Signature Verification

Upon receiving a Template Apply Request for a template whose "syncPubKeyDomain" is set, the DNS Provider MUST perform the following steps and MUST reject the request if any step fails:

1. Check for required parameters - The request MUST carry both "sig" and "key" parameters.
2. Retrieve the public key - Query DNS for TXT records at the host formed by prepending the "key" parameter value as a label to "syncPubKeyDomain" (see [Section 6.4](#)). If no such records are found, the processing fails.
3. Reassemble key fragments - Collect all TXT records at that host, parse each as a "dc-pubkey-record", and concatenate the "d" values in ascending order of "p" to obtain the complete encoded public key.
4. Determine algorithm and key format - Read the "a" and "t" fields. If absent, assume "RS256" and "x509" respectively.
5. Decode the public key - decode the key from Base64 encoding and key format indicated by "t".
6. Construct the canonical input string - Remove the "sig" and "key" key/value pairs from the received query string without reordering or re-encoding the remaining of the query string. If the canonical string needs to be reconstructed from individual parameters (what is not RECOMMENDED), the same construction steps as the signing procedure MUST be followed (see [Section 8.3.2.1](#)).
7. Verify the signature - URL-decode then base64-decode per [\[RFC4648\]](#) the "sig" parameter value, and verify the signature against the input string using the retrieved public key and identified algorithm.

8.3.5. Template Apply Response

After successful processing of Template Apply Request the DNS Provider MUST terminate the user flow according to one of the following cases:

- If "redirect_uri" was present in the request and the template's "syncRedirectDomain" field is set, the DNS Provider MUST redirect the user agent to the "redirect_uri".

The following parameters MUST be appended to the "redirect_uri":

- "state" - If a "state" parameter was present in the request, it MUST be echoed back unchanged as "state=" on the redirect URI.
- If "redirect_uri" was not provided, or if "syncRedirectDomain" is not set in the template, the DNS Provider MUST gracefully terminate the user flow by other means (for example, displaying a completion page). When the user agent is a web browser and the DNS Provider's page was opened in a separate window or tab, the DNS Provider SHOULD attempt to close that window or tab.

To prevent open redirects, the DNS Provider MUST NOT redirect the user agent to a "redirect_uri" value whose registered domain is not listed in the template's "syncRedirectDomain" field, unless the request carries a valid digital signature (see [Section 8.3.2.1](#)).

The DNS Provider MAY append parameters beyond those specified above to the "redirect_uri". The Service Provider MUST ignore any parameter on the redirect that it does not recognize. The presence of such a parameter MUST NOT cause the Service Provider to treat an otherwise successful outcome as a failure. The same applies to the error redirect described in [Section 8.3.6](#).

The Service Provider SHOULD verify the outcome via DNS regardless of how the flow terminates (see [Section 8.4](#)).

8.3.6. Template Apply Error Response

If the DNS Provider cannot complete the apply operation - for example because authentication failed, the user does not control the domain, the domain is suspended, or the user explicitly canceled - the DNS Provider MUST signal an error.

If "redirect_uri" is present and the open-redirect constraint is satisfied, the DNS Provider MUST redirect the user agent to the "redirect_uri" with the following parameters appended:

- "error" - REQUIRED on error. The value MUST be one of the error codes defined in Section 4.1.2.1 of [\[RFC6749\]](#): "invalid_request", "unauthorized_client", "access_denied", "unsupported_response_type", "invalid_scope", "server_error", or "temporarily_unavailable".
- "error_description" - OPTIONAL. A developer-oriented plain-text description of the error, intended for developers rather than the end user and therefore RECOMMENDED to be in English. The DNS Provider SHOULD keep descriptions vague where disclosure of internal account or domain state would be inappropriate.

As a RECOMMENDED convention, when the user explicitly cancels the operation and the DNS Provider uses "error=access_denied", the "error_description" value MAY carry the prefix "user_cancel" to allow the Service Provider to distinguish user cancellation from other denial reasons.

- "state" - If a "state" parameter was present in the request, it MUST be echoed back unchanged as "state=" on the redirect URI.

If "redirect_uri" is not present or the open-redirect constraint is not satisfied, the DNS Provider MUST gracefully terminate the user flow and SHOULD present an appropriate error indication to the user. When the user agent is a web browser and the DNS Provider's page was opened in a separate window or tab, the DNS Provider SHOULD attempt to close that window or tab.

8.4. Verification of Changes

After the synchronous flow completes, the Service Provider SHOULD verify that the expected DNS records are present in the zone by querying the authoritative DNS server for the domain.

DNS verification MUST be treated as the authoritative signal of success. A "redirect_uri" callback received without an "error" parameter does not constitute proof that the template was applied. Users can modify redirect URIs in the user agent, so a successful-looking redirect MUST NOT be relied upon as confirmation of DNS changes. Similarly, a "redirect_uri" callback received with an "error" parameter does not constitute proof that the template was NOT applied. The DNS Provider MAY have applied the template before the error condition arose.

Receipt of a protocol-level completion signal MAY be used as a trigger to initiate DNS verification. However, the Service Provider MUST account for DNS propagation delay and MUST implement a retry mechanism with appropriate intervals until the expected records are observed or a timeout is reached.

To minimize delays caused by DNS resolver caching, it is RECOMMENDED that the Service Provider query a DNS resolver configured with low TTL overrides, or query the authoritative name servers directly.

9. Resolving Template Variables

9.1. Variables

9.1.1. Variable Syntax

Variable expressions are the parameterized parts of a Domain Connect Template. Each expression contains one variable specifier (which can be either a named variable or a special variable "@") that is replaced with a value during template application.

Variable expressions MUST conform to "variable-expression" syntax (see [Section 3](#)).

9.1.2. Special and Built-In Variables

There are three Built-In variables:

- "%host%": This is the host passed from the query string
- "%domain%": This is the domain passed from the query string
- "%fqdn%": This is the fully qualified domain name of template application e.g. [host.]domain

For example, with the query string "domain=example.com&host=", "%fqdn%" in a template would be "example.com", and with "domain=example.com&host=sub1", "%fqdn%" in a template would be "sub1.example.com".

The "@" variable has special meaning, and can be used in the "host"/"name" field or in the "pointsTo" or "target" field in isolation. For the "host"/"name", "pointsTo" and "target" fields it is a shortcut for the value "%fqdn%.". The trailing dot here is equal to the DNS master file notation [RFC1035], which indicates the value is absolute. For some record types, like "A" or "AAAA" usage of "@" in pointsTo would not render a valid IP address, therefore MUST NOT be used. Likewise in "NS" record types it would render a circular delegation therefore MUST NOT be used.

Aside from the built-in variables above and "@", a template variable-name MUST NOT collide with any reserved apply parameter name - the dc-predefined-param names or names beginning with a reserved extension prefix ("dc.", "vnd.") (see [Section 3](#)). These names are reserved for protocol use.

9.2. Variable substitution

9.2.1. Input Values

- Template fields - the string-valued fields of each template record where variables are allowed. Fields MUST be decoded from JSON string encoding before variable substitution is performed.
- Named variable values - the name/value pairs supplied by the caller in the apply request (query string parameters or JSON body). Values MUST be treated as strings regardless of how the underlying data source represents them. Any transport encoding (e.g., URL percent-encoding) MUST be decoded before substitution. The resulting substituted value MUST reflect the exact original input value string.
- Built-in variable values - the values of "%domain%", "%host%", and "%fqdn%" derived from the "domain" and "host" apply parameters (see [Section 9.1.2](#)).

9.2.2. Processing

When a template is applied, the variables in the template are replaced with the values passed as input.

Variables are identified by their name ("variable-name" part of the syntax or "@") and the whole variable expression is replaced with a value of either Built-In Variable or Name/Value pairs provided to the Apply operation.

Only active records (as determined by group filtering, see [Section 10.3](#)) MUST be processed. Template fields of inactive records MUST NOT be processed.

Variables are only allowed in template fields of type string, therefore the input field values from the template MUST be decoded from JSON string encoding before variable substitution.

Variables are replaced in the template fields in the order they are found. If a variable is not found in the input, the processing MUST fail. After a variable is replaced, only the remaining string is used for further variable substitution.

The result of the processing MAY still contain strings containing variable expressions coming from Input Values of variable substitution. The processing MUST NOT fail in this case, and the variable expressions MUST be left as is without any further processing.

The DNS Provider MUST ignore any request parameter not referenced in the template.

9.3. Host Name Rendering

After variable substitution, each template record's "host" or "name" field value is combined with the "domain" and "host" apply parameters to produce the final DNS owner name for the resource record.

9.3.1. Input Values

- "domain" - the Zone Apex, as supplied in the apply request.
- "host" - the Sub Domain label(s), as supplied in the apply request. An empty value or absence of this parameter indicates the Zone Apex.
- The rendered "host" or "name" field value of the template record after variable substitution.

9.3.2. Processing

If the rendered template field value is "@" or empty, it resolves to the root of the applied scope - equivalent to the fully qualified "[host.]domain.".

If the rendered template field value ends with a "." (trailing dot), it is treated as an absolute DNS name and used as-is, without further qualification.

Otherwise, the rendered value is treated as relative and prepended as a label to the fully qualified "[host.]domain" formed by the apply parameters.

9.3.3. Examples of host processing

EXAMPLE: Template records example

```
[{
  "type": "CNAME",
  "host": "www",
  "pointsTo": "@",
  "ttl": 1800
},
{
  "type": "A",
  "host": "@",
  "pointsTo": "192.0.2.1",
  "ttl": 1800
}]
```

DNS zone after the template applied with "domain=example.com" and "host" parameter missing or empty:

```
www 1800 IN CNAME example.com.  
@ 1800 IN A 192.0.2.1
```

alternatively

```
www.example.com. 1800 IN CNAME example.com.  
example.com. 1800 IN A 192.0.2.1
```

DNS zone after the template applied with "domain=example.com" and "host=bar":

```
www.bar 1800 IN CNAME bar.example.com.  
bar 1800 IN A 192.0.2.1
```

alternatively

```
www.bar.example.com. 1800 IN CNAME bar.example.com.  
bar.example.com. 1800 IN A 192.0.2.1
```

9.4. SPF Record Merging

The challenge with SPF [RFC 7208](#) [RFC7208] records and Domain Connect is that an individual service might recommend an SPF record. If only one service were active, this would be accurate. But with several services together only the DNS Provider is able to determine the valid shape of a SPF TXT record.

One solution to this problem is to merge all related records. At the highest level, this means taking everything between the "v=spf1" and the "all" from each of the records and merging them together, terminating with a hard-coded modifier on "all" at the end. For an SPF record to fulfill its purpose of protection against malicious email delivery, it is RECOMMENDED to use a fixed modifier "~" advising lower rating of the messages from other sources not specified in SPF, however DNS Provider MAY set this modifier at their own discretion and according to the current best practice.

```
@ TXT v=spf1 include:spf.mailer1.example include:_spf.newsletter.exam  
ple ~all
```

The other would be to write intermediate records, and reference these locally.

```
r1.example.com. TXT v=spf1 include:spf.mailer1.example ~all  
r2.example.com. TXT v=spf1 include:_spf.newsletter.example ~all  
@ TXT v=spf1 include:r1.example.com include:r2.example.com ~all
```

There are advantages and disadvantages to both approaches. SPF records have a limit of 10 DNS lookups and record length is limited to 255 characters. So depending on the embedded records both approaches might have advantages.

The implementation would be left to the DNS Provider, but to facilitate this SPF records SHOULD NOT be included in templates. Instead, a Computed Record Type (see [Section 3](#)) is introduced in the template called "SPFM". This has the following attribute:

"spfRules": Determines the desired rules, basically everything but leading "v=spf1" and trailing "all" rule - see: [SPF Rules \(Section 6.2\)](#)

When a template is added or removed with an "SPFM" record in the template, some code would need to take the aggregate value of all "SPFM" records in all templates applied as well as existing SPF TXT record on the host and recalculate the resulting SPF TXT record. In case several sources specify the same rule with a different policy DNS Provider SHOULD apply the least restrictive one as a result. "soft failure" SHOULD be preferred over "hard failure", "neutral" SHOULD be preferred over "soft failure".

DNS Provider SHOULD also allow the end user to modify the SPF record after merging.

Due to merging step in between, the resulting SPF TXT records are considered non-essential. That means the user may decide to override the final calculated value or remove the whole SPF record. This action MUST NOT lead to removal of any related templates in conflict detection and template integrity routines if implemented by the DNS Provider.

If the existing TXT record makes the merging operation not possible, the DNS Provider MUST handle this situation the same way as a conflict and either let the end-user resolve it in the UX.

Service Providers MUST NOT check content of TXT SPF record for an exact match, as it might be strongly influenced by the DNS Provider merging strategy and user actions.

See [Appendix A.7](#).

10. Template Application

10.1. Template State in DNS Providers system

DNS Providers MAY choose to maintain state inside records in DNS indicating the templates writing the records.

A DNS Provider that maintains this state may be able to provide an improved experience for users, telling them the services enabled. They also may be able to have more advanced handling of conflicts and assure integrity of applied template instances.

A template instance is identified by the pair "domain" and "host" where the template has been applied. Therefore, the same template MUST be able to be applied more than once to the same DNS zone identified by "domain", as long as the value of "host" parameter differs. Applying the

same template again with the same "host" replaces the previous instance: its records are removed and the records of the new instance are written, unless the rules of [Multi-Instance](#) or [Group Filtering](#) apply. A DNS Provider MAY implement this as an in-place update, provided the resulting zone content and template state are the same as removing and then applying.

Manual changes made by the user at the DNS Provider MAY also trigger conflict detection against applied templates in order to maintain integrity.

10.2. Steps to Apply a Template

The following steps give an overview of the template application process. Each step is described in detail in the sections below.

1. Verify template applicability - For synchronous flow requests, the DNS Provider MUST verify that the template's "syncBlock" field is not "true". If it is, the DNS Provider MUST NOT process the request and MUST return an error.
2. Verify zone ownership - The DNS Provider MUST verify that the target zone identified by "domain" is present in the user's account and that the user is authorized to make changes to it.
3. Filter records to apply - select records applicable for further processing based on "groupId" property of the template and "groupId" apply parameter (see [Section 10.3](#)).
4. Resolve variables - All variable expressions in active template records are substituted with the values provided by the caller, producing the Rendered Record Set: a concrete set of DNS resource records (RRs) to be applied (see [Section 9.2](#)).
 - a. abort if rendered RRs are not possible to be applied to the target zone
5. Perform conflict detection - The DNS Provider checks the Rendered Record Set against the existing zone content according to the conflict detection rules (see [Section 10.4](#)).
 - a. Identify individual zone records that conflict with the Rendered Record Set.
 - b. (only DNS Providers tracking template state): Check each resolved RR against essential records of previously applied templates. Identify which previously applied templates own conflicting records and would be removed in their entirety (see [Section 10.4.1.2](#)).
 - c. (only DNS Providers tracking template state): Mark instances of the same template applied on the same "host" for removal unless "multiInstance" is set for the template (see [Section 10.4.1.1](#)) or "groupId" parameter is supplied in the template application (see [Section 10.3](#)).
6. Pre-calculate conflict resolution - Determine the full set of records and, where applicable, template instances that would be removed upon application (see [Section 10.5](#)).
7. Obtain user authorization - The DNS Provider presents the intended changes and the pre-calculated conflict consequences to the user and obtains explicit authorization before proceeding (see [Section 10.6](#)).
 - a. abort if authorization not granted

8. Apply changes - After authorization is granted:

- a. Remove all records pre-calculated for removal from the zone.
- b. Write the Rendered Record Set to the zone in totality.
 - i. Note: if the Rendered Record Set yields differing "ttl" values across new and existing RRs that form the same RRset, the DNS Provider MUST reconcile them to a single TTL (e.g., by taking the lowest value), per [\[RFC2181\]](#) Section 5.2
- c. (only DNS Providers tracking template state): Record the new template instance for the "domain" and "host" pair and remove the state of any template instances that were resolved for removal.

10.3. Group Filtering

Before variable substitution and any further processing, the DNS Provider MUST determine the active record set - the subset of template records that are subject to processing - by applying the following rules to each record in the template:

1. A record with no "groupId" field is always active, regardless of whether a "groupId" parameter was supplied in the apply request.
2. A record with a "groupId" field is active if and only if its "groupId" value appears in the list supplied as the "groupId" apply parameter. Matching is case-sensitive and exact.
3. If no "groupId" parameter is supplied in the apply request, all records are active irrespective of whether they carry a "groupId".

Only active records MUST be subject to variable substitution, conflict detection, and zone write operations. Inactive records MUST be excluded from all template processing steps and MUST NOT be written to the zone.

If the "groupId" parameter is supplied but no record in the template carries a "groupId" that matches any of the specified identifiers (i.e., the active record set consists solely of ungrouped records or is empty), the DNS Provider MUST return an error and MUST NOT make any changes to the zone.

10.3.1. Group Filtering and Template State

When a "groupId" parameter is supplied, the apply operation is additive: only the records belonging to the specified groups are written, while records of other groups that were written by a prior application of the same template instance MUST NOT be disturbed.

Consequently, for DNS Providers that maintain applied template state, an existing instance of the same template on the same "domain" and "host" MUST NOT be removed before the new records are written, regardless of conflict-detection outcome for the active record set, however such provider MUST remove the records previously written for the same group(s) before writing the new active record set, in order to avoid accumulation of stale records from superseded group applications.

10.4. Conflict Detection

Conflict detection is performed by the DNS Provider prior to template application. The rules below ensure predictable conflict resolution between DNS Providers. Each rule applies to records on the same host, unless otherwise specified.

- A CNAME record conflicts with any other record on the same host, including another CNAME, and any existing records conflict with a CNAME.
- An NS record conflicts with all other records on the same host, and with any record whose host is subordinate to the NS host. For example, an NS record for "foo" conflicts with any record at "foo", "www.foo", "bar.foo", etc. Conversely, any other record type conflicts with NS records in the same manner.
- MX and SRV records conflict with any other record of the same type on the same host.
- A and AAAA records conflict with any other A or AAAA record on the same host, to avoid IPv4 and IPv6 addresses pointing to different services.
- TXT record conflict detection is governed by the "txtConflictMatchingMode" property of the template record:
 - "None" - the record does not conflict with any other TXT record. This is the default.
 - "All" - the record conflicts with any other TXT record on the same host.
 - "Prefix" - the record conflicts with any other TXT record on the same host whose value starts with "txtConflictMatchingPrefix".

Note: A DNS Provider MUST also check the applicability of all generated records to the target DNS zone against constraints imposed by the DNS protocol itself, such as the coexistence rules of [\[RFC2181\]](#), as well as any other local capabilities, limits or policies.

10.4.1. Conflict Detection Special Handling

10.4.1.1. Multi-Instance

This processing is only REQUIRED for DNS Providers that maintain applied template state.

By default a template is expected to be applied only once to a target "domain" and "host", so applying the same template again replaces the previous instance (see [Section 10.2](#)). For some templates however it is desirable to be able to create multiple instances (for example add second TXT record on the same host as opposed to replacing the current value).

A template flag [multiInstance](#) ([Section 6.1](#)) can be set in order to allow for that. This tells the DNS Provider that the template is expected to be written multiple times and that a re-apply MUST NOT remove previous instances with the same "host". Regular conflict detection rules MUST still be processed between instances, therefore such a template MUST be designed so that it does not conflict with itself.

10.4.1.2. Essential Records

This processing is only REQUIRED for DNS Providers that maintain applied template state.

A template record is considered essential when it **MUST** be present and remain consistent with the applied template for the entire lifetime of the service. The essential property of a template record controls this behavior and **MUST** be set to "Always" (the default) for such records.

A DNS Provider that maintains applied template state **MUST** treat an attempt of removal or modification of such an essential record as a conflict indicator, and **MUST** require the full template to be removed or abort.

Records with "essential=OnApply" **MUST** be applied when the template is first applied, but **MAY** be subsequently removed or altered - by the user or through the application of another template - without triggering conflict or removal of the owning template.

10.4.1.3. Identical Records

A resolved template record that is byte-identical to a resource record already present in the target zone - same owner name, type, class and RDATA - **MUST NOT** be treated as a conflict, regardless of the conflict-detection rules above. Instead of removing the existing record or the whole template, the DNS Provider applies the new template on top of it, so that the record is now shared between the existing configuration and the newly applied template.

Whether a second, byte-identical copy of the record is stored in DNS Provider's system, or the duplicate is skipped so that a single record remains, is left to the DNS Provider and the capabilities of its underlying DNS software. Both behaviors are conformant - the observable DNS response **MUST** be equivalent in either case and contain only a single resource record.

10.5. Calculating Conflict Resolution

A DNS Provider not maintaining applied template state, applying a template whose records conflict with any existing record in the zone **MUST** remove all those conflicting records.

When a DNS Provider maintains applied template state, applying a template whose records conflict with records written by a previously applied template **MUST** cause the conflicting prior template to be removed in its entirety, unless a conflicting record is explicitly marked as non essential and eligible for removal. As an example: if template T1 wrote records A and B, and template T2 is applied with records B and C, record B conflicts, and T1 is removed together with all its records before T2 is applied. Where such a provider shares a single record between more than one applied template instance (see [Section 10.4.1.3](#)), it **MUST** retain that record until the last instance referencing it is removed, and **MUST** remove it only when no remaining instance still requires it.

10.6. User Authorization of Changes

It is ultimately left to the DNS Provider to determine the amount of disclosure and/or conflict detection.

The DNS Provider MUST inform the user of the service that is about to be enabled when requesting authorization. If the user wishes to review the specifics, the DNS Provider SHOULD make the individual DNS records being set available, in order to allow informed decision before changes are applied.

If conflicts are detected - at either the template or record level - the DNS Provider SHOULD present these to the end user before authorization is granted - otherwise the end user may fail to recognize the whole scope and consequence of the change. For template instances this means identifying services that would be disabled. For records this means identifying records that would be deleted or overwritten.

The DNS Provider MUST allow the user to override a detected conflict and proceed with the template application, or to abort the process.

When presenting the authorization request to the user, the DNS Provider SHOULD display the template's "providerName" and "serviceName" fields. When a caller supplies "providerName" or "serviceName" request parameters (permitted only when the template's "sharedProviderName" or "sharedServiceName" flags are set, respectively), the DNS Provider SHOULD use those values to augment the displayed name. It is RECOMMENDED however to also display the original "providerName" and "serviceName" defined in the template.

Unless the apply request is digitally signed (see [Section 8.3.2.1](#)), the DNS Provider MUST treat the template as susceptible to phishing: it SHOULD display additional warnings prompting the user to verify the source of the request before proceeding, and SHOULD provide a more verbose description of the changes about to be applied. Failure to do so may expose the end user to the risk of malicious changes being applied to the DNS zone.

10.7. Template Application

In the Synchronous Flow, after authorization is granted, the DNS Provider applies the template to the DNS zone.

The DNS Provider MUST remove all conflicting templates and records from the target zone.

The DNS Provider MUST apply all records of the template in totality as one atomic operation.

10.8. Examples

Examples of template processing are in [Appendix A](#).

11. Security Considerations

11.1. Template Variable Phishing

Templates that accept variable parameters introduce a phishing risk. The more static the values in a template record, the more secure the template. When variable values cannot be avoided, a bad actor could craft an apply URL substituting a malicious value - for example, a hijacked IP

address - and send it to users as a seemingly legitimate reconfiguration request. The user would be presented with a DNS Provider consent screen that appears valid, but would result in DNS being pointed at infrastructure under the attacker's control.

Not all templates are susceptible. The risk is proportional to how much of a record's content is controlled by variables. Two mitigations are available to template authors and Service Providers: digitally signing apply requests (see the Section <<[Section 8.3.2.1](#),format=title>>), and using an alternative flow not prone to this risk by disabling the synchronous flow via "syncBlock". In the absence of such a mitigation a template is treated as susceptible to phishing by default, and DNS Providers display the additional warnings described in [Section 10.6](#).

11.2. Untrusted DNS Provider Discovery Records

DNS Provider Discovery (see [Section 7](#)) requires the Service Provider to read the "_domainconnect" TXT record from the target zone and to fetch the Settings End-Point at the URL derived from it. The content of this record is controlled by whoever controls the zone, which is not necessarily a party the Service Provider trusts.

A malicious or compromised zone can therefore direct the Service Provider's HTTP client at an arbitrary endpoint. Two classes of attack arise:

- Denial of Service - The record may point at a resource that is malformed, never terminates, responds very slowly, or returns an oversized or unbounded body, with the goal of exhausting the Service Provider's connections, memory, or processing capacity.
- Server-Side Request Forgery (SSRF) - The derived authority may resolve to an address inside the Service Provider's own infrastructure (for example a loopback, link-local, or private-use address, or an internal-only hostname), causing the Service Provider to issue requests against internal services on the attacker's behalf.

Service Providers MUST treat the "_domainconnect" TXT record content, and the Settings End-Point response derived from it, as untrusted input. In particular, Service Providers:

- SHOULD reject derived authorities that resolve to loopback, link-local, private-use, or otherwise internal addresses, unless the Service Provider has a specific reason to permit them
- MUST bound the discovery request, including connection and total-request timeouts, a maximum response size, and a limit on the number of redirects followed

11.3. Underscored Host Names

The "host" apply parameter is supplied by the requesting party and prepended to the template record owner names during [Host Name Rendering](#). When its value is an underscore-prefixed label per [\[RFC8552\]](#), the applied records land under a name with special protocol meaning - for example "_acme-challenge", "_domainkey", or "_dmarc". A template that places a "CNAME" on the apex ("@") host could then be applied with such a "host" value to redirect one of these

underscored names to infrastructure under an attacker's control, causing the user to authorize what appears to be an ordinary change while in fact delegating certificate issuance, DKIM key publication, or DMARC policy.

Legitimate uses of an underscored "host" value exist but are rare. DNS Providers SHOULD therefore apply additional measures when the "host" apply parameter contains an underscore-prefixed label, such as presenting the affected owner names to the user in a more explicit and verbose manner during [User Authorization of Changes](#) so that the special-meaning target is not obscured.

11.4. Must-Understand Markers and Pre-Existing Deployments

The "crit" must-understand marker (see [Section 4.3](#)) is effective only against implementations that recognize "crit" itself. Deployments predating this specification, including those implementing [\[DC-SPEC\]](#) alone, ignore unrecognized elements without inspecting "crit", and therefore proceed as though no such marker were present.

A sender consequently cannot infer from the absence of a rejection that the receiver honored a "crit" marker. Where an extension conveys a security property - for example a constraint narrowing what a template may modify, or an assertion the receiver is expected to enforce - marking it in "crit" does not by itself prevent a receiver that predates the extension from applying the request without that property. "crit" MUST NOT be treated as a downgrade-prevention mechanism, and the security of an extension MUST NOT rest on the receiver's recognition of "crit" alone.

11.5. Template Onboarding

Templates reach a DNS Provider out of band. This protocol has no mechanism by which a DNS Provider retrieves a template from a Service Provider. A DNS Provider therefore cannot infer a template's provenance from the protocol, and the trust decision is made entirely during the onboarding process described in [Section 4.2](#).

Two classes of attack follow. In the first, an attacker submits a template under its own identity whose records are broader than the service it claims to provide, or accepts variables in positions that let the applied value be chosen freely at apply time. In the second, an attacker submits a template impersonating an established Service Provider, borrowing a recognizable "providerName", "serviceName", or "logoUrl" so that the consent screen shown during [User Authorization of Changes](#) carries the reputation of the party being impersonated. The "providerId" and "serviceId" fields are self-asserted identifiers and are not validated by any authority.

DNS Providers therefore treat template onboarding as a trust decision rather than a data import, and:

- MUST verify that the party submitting a template controls the identity it claims, rather than relying on the identifying fields carried in the template. Where the template specifies

"syncPubKeyDomain", demonstrated control of that domain and of the key published under it (see [Section 6.4](#)) serves this purpose.

- MUST review the records a template requests before accepting it, and reject or require further justification for a scope wider than the service being offered.
- MUST confirm that the display fields presented to users, in particular "providerName", "serviceName", and "logoUrl", are not liable to be confused with those of another Service Provider already onboarded.
- MUST re-apply these checks when an onboarded template is updated, since a subsequent "version" of an accepted template may widen its scope or alter its display fields.

12. IANA Considerations

12.1. Registration Procedure for Domain Connect Registries

IANA is requested to create a new registry group named "Domain Connect Protocol". The registration procedure described in this section applies to all registries established by this document under that registry group.

IANA is requested to appoint one or more designated experts for the registries in the "Domain Connect Protocol" registry group. Multiple designated experts are preferred, able to represent the perspectives of different Service Providers and DNS Providers, so that registration decisions benefit from broadly informed review. Where a registration decision could be perceived as creating a conflict of interest for a particular expert, that expert is expected to defer to the judgment of the other experts.

The following guidance is offered to the designated experts. It describes how to decide whether to approve a registration request.

The primary consideration is whether the request contains enough information to populate the registry and whether the referenced specification is clear enough to allow interoperable implementations. Experts are expected to be lenient, approving requests that meet this bar, and to ask for corrections rather than reject a request that is merely incomplete.

Grounds for asking for corrections or refusing a request include:

- the registration request or the referenced specification does not describe the registered value in enough detail for an implementer to interoperate;
- the proposed value duplicates functionality that already exists in the registry, without explaining why a separate value is needed;
- the proposed value does not conform to the syntax required for that registry;
- the proposed value would change the meaning of an existing entry, in a way that is incompatible with its existing use;
- the proposed value would occupy a term, which is not corresponding to the described use, or is too generic for the specific use case.

To allow values to be allocated before the referenced specification is published, the experts may approve a registration once they are satisfied that the specification will be published.

Change control for the registries in this group - that is, altering a registry's format or registration policy, or reassigning the meaning of an existing entry - requires either a new Standards Track RFC updating this document or IESG Approval, per [\[RFC8126\]](#) Section 4.9. This is distinct from the registration policy of each registry, which governs new entries only.

12.2. Domain Connect Fully Specified Record Types Registry

IANA is requested to create a new registry named "Domain Connect Fully Specified Record Types" under the "Domain Connect Protocol" registry group.

Registration policy: Specification Required [\[RFC8126\]](#).

Each entry in the registry includes:

- **RR Type**: the record type name as it appears in a template record "type" field. The value MUST conform to the dc-type-name syntax (see [Section 3](#)).
- **Status**: the lifecycle state of the registration. The following values are defined. The designated experts may define additional values as needed:
 - "Active": the type is currently in use and its definition is normative.
 - "Deprecated": the type SHOULD NOT be used in new templates. It MAY appear in existing deployments for backwards compatibility. If the corresponding DNS RR TYPE is marked deprecated or obsolete in the IANA "Resource Record (RR) TYPES" registry, the entry here SHOULD be set to "Deprecated".
 - "Reserved": the type appears in earlier versions of the Domain Connect specification (see [\[DC-SPEC\]](#)) and is reserved for back-compatibility. It MUST NOT be redefined in a manner incompatible with [\[DC-SPEC\]](#), and MAY only be defined by a specification that extends this document. See [Section 3, Paragraph 13](#).
- **Handling**: how the DNS Provider processes a record of this type, as defined in [Section 3](#). The value MUST be one of "Fully Specified" or "Computed". Generic Record Types are carried opaquely and are not registered here. A "Reserved" entry MAY leave this field unassigned.
- **DNS RR TYPE**: the name of the corresponding entry in the IANA "Resource Record (RR) TYPES" registry within the "Domain Name System (DNS) Parameters" registry group. A "Fully Specified" type normally corresponds to a registered DNS RR TYPE of the same name. Before registering such a type, the designated experts verify that the named DNS RR TYPE exists in that registry. For a "Computed" type, a "Reserved" type, or any other type for which no corresponding DNS RR TYPE exists or would be meaningful, the value MUST be "N/A". Where a "Fully Specified" type would use a name that is not yet a registered DNS RR TYPE, or where an entry is registered with a "DNS RR TYPE" of "N/A" (such as a "Computed" type), the designated experts are expected to coordinate with the designated experts of the "Resource Record (RR) TYPES" registry before completing the registration, to confirm that the chosen name does not collide with an existing or anticipated DNS RR TYPE.

- **Kind:** the nature of the defining document. The value **MUST** be one of: "IETF Standard" for types defined in a standards-track RFC, "Informational" for types defined in an Informational RFC, or "Other" for any other specification.
- **Reference:** the document that defines the type.

A registration request **MUST** provide the following:

RR Type: The record type name (the "type" value), conforming to the dc-type-name syntax (see [Section 3](#)).

Status: One of "Active", "Deprecated", or "Reserved".

Handling: "Fully Specified" or "Computed"; omitted for a "Reserved" entry.

DNS RR TYPE: The name of the corresponding IANA DNS RR TYPE, or "N/A".

Kind: "IETF Standard", "Informational", or "Other".

Reference: The document that defines the type, preferably including a URI from which a copy can be retrieved.

The following types from this specification constitute the initial contents of the registry:

RR Type	Status	Handling	DNS RR TYPE	Kind	Reference
"A"	Active	Fully Specified	"A"	IETF Standard	This document
"AAAA"	Active	Fully Specified	"AAAA"	IETF Standard	This document
"CNAME"	Active	Fully Specified	"CNAME"	IETF Standard	This document
"MX"	Active	Fully Specified	"MX"	IETF Standard	This document
"TXT"	Active	Fully Specified	"TXT"	IETF Standard	This document
"SRV"	Active	Fully Specified	"SRV"	IETF Standard	This document
"NS"	Active	Fully Specified	"NS"	IETF Standard	This document

RR Type	Status	Handling	DNS RR TYPE	Kind	Reference
"SPFM"	Active	Computed	"N/A"	IETF Standard	This document
"APEXCNAME"	Reserved	Computed	"N/A"	Other	[DC-SPEC]
"REDIR301"	Reserved	Computed	"N/A"	Other	[DC-SPEC]
"REDIR302"	Reserved	Computed	"N/A"	Other	[DC-SPEC]

Table 4: Initial Domain Connect Fully Specified Record Types

12.3. Domain Connect Settings Properties Registry

IANA is requested to create a new registry named "Domain Connect Settings Properties" under the "Domain Connect Protocol" registry group.

Registration policy:

- For property names without any prefix: Specification Required [RFC8126]. Registration additionally requires a standards-track RFC that normatively requires this document as its base.
- For property names beginning with "vnd. ": Specification Required [RFC8126].

Each entry in the registry includes:

- Property name: the JSON key as it appears in the settings response. The value MUST conform to the dc-settings-property syntax (see Section 3).
- Status: the lifecycle state of the registration. The following values are defined. The designated experts may define additional values as needed:
 - "Active": the property is currently in use and its definition is normative.
 - "Deprecated": the property SHOULD NOT be used in new implementations. It MAY appear in existing deployments for backwards compatibility.
 - "Reserved": the property appears in earlier versions of the Domain Connect specification (see [DC-SPEC]) and is reserved for back-compatibility. It MUST NOT be redefined in a manner incompatible with [DC-SPEC]. See Section 3, Paragraph 13.
- Kind: the nature of the defining document. The value MUST be one of: "IETF Standard" for properties defined in a standards-track RFC, "Informational" for properties defined in an Informational RFC, or "Other" for any other specification.
- Reference: the document that defines the property.

A registration request MUST provide the following:

Property name: The JSON key as it appears in the settings response.

Status: One of "Active", "Deprecated", or "Reserved".

Kind: "IETF Standard", "Informational", or "Other".

Reference: The document that defines the property, preferably including a URI from which a copy can be retrieved.

The following properties from this specification constitute the initial contents of the registry:

Property Name	Status	Kind	Reference
"providerId"	Active	IETF Standard	This document
"providerName"	Active	IETF Standard	This document
"providerDisplayName"	Active	IETF Standard	This document
"urlSyncUX"	Active	IETF Standard	This document
"urlAsyncUX"	Reserved	Other	[DC-SPEC]
"urlAPI"	Active	IETF Standard	This document
"width"	Reserved	Other	[DC-SPEC]
"height"	Reserved	Other	[DC-SPEC]
"urlControlPanel"	Reserved	Other	[DC-SPEC]
"nameServers"	Active	IETF Standard	This document
"crit"	Active	IETF Standard	This document

Table 5: Initial Domain Connect Settings Properties

12.4. Domain Connect Template Properties Registry

IANA is requested to create a new registry named "Domain Connect Template Properties" under the "Domain Connect Protocol" registry group (see [Section 12.3](#)).

Registration policy:

- For property names without any prefix: Specification Required [\[RFC8126\]](#). Registration additionally requires a standards-track RFC that normatively requires this document as its base.
- For property names beginning with "vnd. ": Specification Required [\[RFC8126\]](#).

This registry covers the properties of the template definition (see [Section 6.1](#)).

Each entry in the registry includes:

- **Property name:** the JSON key of the property as it appears in the template definition. The value **MUST** conform to the `dc-template-property` syntax (see [Section 3](#)).
- **Status:** the lifecycle state of the registration. The following values are defined. The designated experts may define additional values as needed:
 - **"Active"**: the property is currently in use and its definition is normative.
 - **"Deprecated"**: the property **SHOULD NOT** be used in new templates. It **MAY** appear in existing templates for backwards compatibility.
 - **"Reserved"**: the property appears in earlier versions of the Domain Connect specification (see [\[DC-SPEC\]](#)) and is reserved for back-compatibility. It **MUST NOT** be redefined in a manner incompatible with [\[DC-SPEC\]](#). See [Section 3, Paragraph 13](#).
- **Kind:** the nature of the defining document. The value **MUST** be one of: **"IETF Standard"** for properties defined in a standards-track RFC, **"Informational"** for properties defined in an Informational RFC, or **"Other"** for any other specification.
- **Reference:** the document that defines the property.

The following properties from this specification constitute the initial contents of the registry:

Property Name	Status	Kind	Reference
"providerId"	Active	IETF Standard	This document
"providerName"	Active	IETF Standard	This document
"serviceId"	Active	IETF Standard	This document
"serviceName"	Active	IETF Standard	This document
"version"	Active	IETF Standard	This document
"templateSpecVersion"	Active	IETF Standard	This document
"logoUrl"	Active	IETF Standard	This document
"description"	Active	IETF Standard	This document
"variableDescription"	Active	IETF Standard	This document
"syncBlock"	Active	IETF Standard	This document
"shared"	Deprecated	IETF Standard	This document
"sharedProviderName"	Active	IETF Standard	This document
"sharedServiceName"	Active	IETF Standard	This document

Property Name	Status	Kind	Reference
"syncPubKeyDomain"	Active	IETF Standard	This document
"syncRedirectDomain"	Active	IETF Standard	This document
"multiInstance"	Active	IETF Standard	This document
"warnPhishing"	Deprecated	IETF Standard	This document
"hostRequired"	Active	IETF Standard	This document
"crit"	Active	IETF Standard	This document
"records"	Active	IETF Standard	This document

Table 6: Initial Domain Connect Template Properties

12.5. Domain Connect Protocol-Level Record Fields Registry

IANA is requested to create a new registry named "Domain Connect Protocol-Level Record Fields" under the "Domain Connect Protocol" registry group (see [Section 12.3](#)).

This registry covers only those template record fields that are protocol-level, that is, fields whose meaning is defined by this protocol and that are applicable to a template record irrespective of its "type" (see [Section 6.2.1](#)). Fields that carry the RDATA of a particular record type, or that are otherwise meaningful only for specific record types, are not registered here: the "type" field partitions that part of the name space, and each record type is free to define its own type-specific fields in the specification that defines the type.

Because a protocol-level field applies to records of every type, it occupies its name across all record types at once. A specification registering a protocol-level field MUST therefore establish that the requested name does not collide with a type-specific field name already in use by any record type, whether or not that type-specific field is itself registered. A name that collides in this way MUST NOT be registered, and a different name chosen.

Registration policy:

- For field names without any prefix: Specification Required [[RFC8126](#)]. Registration additionally requires a standards-track RFC that normatively requires this document as its base.
- For field names beginning with "vnd. ": Specification Required [[RFC8126](#)].

Each entry in the registry includes:

- Field name: the JSON key of the field as it appears in a template record. The value MUST conform to the `dc-record-field` syntax (see [Section 3](#)).

- **Status:** the lifecycle state of the registration. The following values are defined. IANA MAY define additional values as needed:
 - **"Active"**: the field is currently in use and its definition is normative.
 - **"Deprecated"**: the field SHOULD NOT be used in new templates. It MAY appear in existing templates for backwards compatibility.
 - **"Reserved"**: the field appears in earlier versions of the Domain Connect specification (see [DC-SPEC]) and is reserved for back-compatibility. It MUST NOT be redefined in a manner incompatible with [DC-SPEC]. See [Section 3, Paragraph 13](#).
- **Kind:** the nature of the defining document. The value MUST be one of: "IETF Standard" for fields defined in a standards-track RFC, "Informational" for fields defined in an Informational RFC, or "Other" for any other specification.
- **Reference:** the document that defines the field.

The following protocol-level record fields from this specification constitute the initial contents of the registry:

Field Name	Status	Kind	Reference
"type"	Active	IETF Standard	This document
"groupId"	Active	IETF Standard	This document
"essential"	Active	IETF Standard	This document
"crit"	Active	IETF Standard	This document

Table 7: Initial Domain Connect Protocol-Level Record Fields

12.6. Guidance to the DNS Resource Record (RR) TYPEs Registry

To avoid collisions between newly assigned DNS RR TYPE mnemonics and the record type names used by Domain Connect, IANA is requested to add the following note to the registration procedure of the "Resource Record (RR) TYPEs" registry within the "Domain Name System (DNS) Parameters" registry group (see [RFC6895]):

When evaluating a request to assign a new RR TYPE mnemonic, the designated experts are expected to consult the "Domain Connect Fully Specified Record Types" registry. If the requested mnemonic matches the name of an entry in that registry whose DNS RR TYPE field is "N/A", the designated experts flag the potential conflict and consult the designated experts of the "Domain Connect Fully Specified Record Types" registry before completing the assignment.

This coupling ensures that a Domain Connect record type name that does not (yet) correspond to a DNS RR TYPE - such as a "Computed" type - is not inadvertently assigned a conflicting meaning in the DNS RR TYPES registry.

In addition, IANA is requested to update the "Reference" of the "Resource Record (RR) TYPES" registry to cite this document alongside the existing references:

OLD: [RFC6895][RFC1035]
NEW: [RFC6895][RFC1035][This document]

This records, on the DNS side, the obligation to consult the "Domain Connect Fully Specified Record Types" registry described above.

12.7. Underscore "_domainconnect" DNS Node Name

Per [RFC8552], please add the following entry to the "Underscored and Globally Scoped DNS Node Names" registry:

RR Type	_NODE NAME	Reference
TXT	_domainconnect	This document.

Table 8: IANA Registration for _domainconnect

12.8. Media Type Registration for application/domainconnect-template+json

IANA is requested to register the following media type in the "Media Types" registry, per [RFC6838].

Type name: application

Subtype name: domainconnect-template+json

Required parameters: N/A

Optional parameters: N/A

Encoding considerations: binary; the content is a JSON [RFC8259] document encoded in UTF-8.

Security considerations: A Domain Connect template is a declarative JSON document. It contains no executable content and no scripting or macro facility, and processing a template does not cause code to be executed on the recipient's system. A template does, however, describe DNS records that a DNS Provider is being asked to add to a user's zone, and applying

one changes the configuration of that domain. A recipient therefore does not act on a template solely because it has been received. Measures are required on DNS Provider side to mitigate risks related to the template and its application (see [Section 11](#) and [Section 10.6](#)).

The format employs no compression, so a template presents no decompression-expansion attack surface, nor does it contain any sensitive content which would require external protection measures.

Interoperability considerations: See this document.

Published specification: This document (see [Section 6.1](#)).

Applications that use this media type: Domain Connect Service Providers and DNS Providers.

Fragment identifier considerations: The "+json" structured syntax suffix is defined in [\[RFC6839\]](#). See also [\[RFC8259\]](#).

Additional information: Deprecated alias names for this type: N/A

Magic number(s): N/A

File extension(s): N/A

Macintosh file type code(s): N/A

Person email address to contact for further information: IETF DCONN Working Group
<<mailto:dconn@ietf.org>>

Intended usage: COMMON

Restrictions on usage: N/A

Author: IETF DCONN Working Group

Change controller: IETF

Implementation Status

This section is to be removed before publishing as an RFC.

DNS Providers

Open Source

- Server library (Python):<https://github.com/Domain-Connect/DomainConnectApplyZone>

Proprietary implementations

- ~20 providers, incl. GoDaddy, IONOS, Cloudflare, Squarespace Domains (formerly Google), WordPress.com or Plesk

- 35% of the .com zone (May'24)

Service Providers

Open Source

- Example service: <https://exampleservice.domainconnect.org/https://github.com/Domain-Connect/exampleservice>
- Client library (Python): https://github.com/Domain-Connect/domainconnect_python

Proprietary implementations

- 492 templates from over 250 providers, incl. O365, Google Workspace, Apple Cloud+, Weebly, Squarespace.

Source: <https://stats.domainconnect.org/>

Acknowledgements

The authors wish to thank the following persons for their feedback and suggestions as well as for the previous work on the standard:

- Roger Carney of GoDaddy Inc.
- Chris Ambler of GoDaddy Inc.
- Darrel Miller
- Peter Thomassen
- Paul Hoffmann
- Arnt Gulbrandsen

Change History

This section is to be removed before publishing as an RFC.

Change from draft-ietf-dconn-domainconnect-03 to -04

- Partitioned the name spaces of apply parameters, template properties, template record fields, and settings properties into standards-track and vendor-private trees, and established IANA registries to coordinate them.
- Defined a common extensibility model for all protocol elements, combining an ignore-unknown default with a "crit" must-understand marker, and illustrated it with examples.
- Aligned the Apply Template URL template with the defined apply parameters and extension parameters.
- Added [Protocol Versioning and Extensibility](#), describing how the protocol is versioned and how future revisions preserve backwards compatibility.
- Aligned the registration policies of all Domain Connect registries with the review process of [Registration Procedure for Domain Connect Registries](#).

- Added [Apply Parameter Extensibility](#) to [Parameters/properties](#), reserving the "dc." and "vnd." prefixes and the unprefixed grandfathered parameter names.
- Reserved built-in and grandfathered apply parameter names as forbidden variable-name identifiers in [Special and Built-In Variables](#).
- Added dc-apply-param, dc-predefined-param, dc-dot-param, and dc-vnd-param to [Terminology](#).
- Added [Template Property Extensibility](#) and the "crit" property to [Template Definition](#).
- Added the "templateSpecVersion" property to [Template Definition](#).
- Added the IANA "Domain Connect Template Properties" registry in [Domain Connect Template Properties Registry](#).
- Added dc-template-property, dc-unprefixed-property, and dc-vnd-property to [Terminology](#).
- Clarified how the language of user-facing and developer-facing protocol text relates to the user agent's language preference.
- Reformulated the IANA registration procedure as guidance to the designated experts, replacing normative keywords and IANA-directed process with advice on when to approve a registration, and separately clarified the change control policy governing the registries themselves.
- Corrected nits from early Dnsdir review.
- Clarified RR placement constraints arising from the DNS protocol.
- Noted RRset TTL consistency requirements in [Template Record](#) and [Steps to Apply a Template](#).
- Introduced Rendered Record Set terminology in [Steps to Apply a Template](#) to disambiguate from RRset.
- Added security considerations for the template format and for template onboarding, covering acceptance of untrusted or impersonating templates.
- Clarified rendering of host, name, pointsTo and target values, including fully qualified names and empty hosts.
- Aligned the template application steps with group filtering, user authorization, re-application of templates, and the removal of the asynchronous flow.

Change from draft-ietf-dconn-domainconnect-02 to -03

- Classified template record types as Fully Specified, Computed, or Generic.
- Added a shared [Registration Procedure for Domain Connect Registries](#) section (designated-expert assignment and review process) governing all Domain Connect registries.
- Coupled the "Domain Connect Fully Specified Record Types" registry to the IANA DNS Resource Record (RR) TYPEs registry as a bidirectional maintenance contract (following the pattern set by RFC 9108 for that registry).
- Added [Untrusted DNS Provider Discovery Records](#) to [Security Considerations](#).
- Clarified handling of multiple "_domainconnect" TXT records in [DNS Provider Discovery](#).

- Deprecated and reserved the "warnPhishing" template flag in [Template Definition](#); phishing warnings are now default behavior, see [Template Variable Phishing](#).
- Clarified one-host-one-key rule in [Public Key Publication](#).
- Added [Media Type Registration for application/domainconnect-template+json](#) in [IANA Considerations](#).
- Added media type identification to [Template Definition](#).
- Added HTTP content negotiation guidance to the Query Supported Template end-point.
- Added [Identical Records](#) to [Conflict Detection](#) and shared-record retention to [Calculating Conflict Resolution](#).
- Added examples illustrating an optional path segment in the "_domainconnect" TXT record content and in the settings-response URL prefixes in [DNS Provider Discovery](#).
- Added [Underscored Host Names](#) to [Security Considerations](#).
- Marked the "width", "height" and "urlControlPanel" [DNS Provider Discovery](#) settings properties as Reserved, deferring their definition to [DC-SPEC]; removed reserved properties from the example settings response.

Change from draft-ietf-dconn-domainconnect-01 to -02

- Removed the Asynchronous OAuth Flow from this document; the synchronous flow is now the sole protocol flow defined here. The asynchronous flow is reserved for a companion extension specification.

Change from draft-ietf-dconn-domainconnect-00 to -01

- Added comprehensive ABNF grammar to the Terminology section covering all protocol identifiers, domain name forms (including IDN), template fields, OAuth parameters, and signing artifacts; replaced informal prose constraints throughout parameter tables and field definitions with normative ABNF references.
- Restructured [Template Application](#): added [Group Filtering](#), Essential Records, Steps to Apply a Template, and focused [Conflict Detection](#), [User Authorization of Changes](#) subsections; moved UX and runtime-processing rules into their appropriate sections; moved [Domain Connect Objects and Templates](#) before [Applying Domain Connect](#) in the document structure.
- Restructured request signing into [Signing Procedure](#), [Signature Verification](#), and [Public Key Publication](#)
- Replaced Sync Apply Interaction with a normative Request/Response/Error structure; rewrote [Verification of Changes](#) as normative text.
- Restructured [Security Considerations](#): moved phishing threat into new [Template Variable Phishing](#) subsection; editorial cleanup.
- Defined extensibility model for [DNS Provider Discovery](#) settings response and added IANA "Domain Connect Settings Properties" registry.
- Removed non-normative sections (Public Template Repository, General Considerations, Extensions/Exclusions); replaced all occurrences of "browser" with "user agent".
- Shortened [Trust Model](#) section.

Change from draft-kowalik-domainconnect-02 to draft-ietf-dconn-domainconnect-00

- DCONN WG adoption. No other changes.

Change from -01 to -02

- Draft refresh from expire. No content changes.

Change from -00 to -01

- Changed term Root Domain to Zone Apex to align with [RFC8499].
- Removed example provider names from Service Providers and DNS Providers terminology
- Added Use Cases
- Added Trust Model
- Added sequence diagrams for synchronous and asynchronous flows instead of UX mocks
- Reviewed use of normative language
- Cleaned up usage of terminology
- Variable substitution description updated
- All URLs are now normatively defined with URI templates

Change from draft-kowalik-regext-domainconnect-00 to draft-kowalik-domainconnect-00

- Added possibility to specify any DNS record type in a generic manner.
- Added possibility to define variables for numeric fields.
- Added IANA registration for _domainconnect record as per [\[RFC8552\]](#)

Change from draft-carney-regext-domainconnect-03 to draft-kowalik-regext-domainconnect-00

- Version synchronized with 2.2 version rev. 66 of the public Domain Connect specification.

Change from -02 to -03

- Added width/height JSON values returned by DNS Provider Discovery.
- Corrected text of GET method for getting the authorization token.
- Added clarifying text to Group ID description parameter of the apply template POST method. Quite a few minor edits and clarifications that were found during implementation, especially in the Implementation Considerations section.

Change from -01 to -02

- Added new GET method for Service Providers to determine if the DNS Provider supports a specific template. Some other minor edits for clarification.

Change from draft-carney-regext-domainconnect-00 to -01

- Minor edits and clarifications found during implementation.

Normative References

- [RFC1035] Mockapetris, P., "Domain names - implementation and specification", STD 13, DOI 10.17487/RFC1035, BCP 13, RFC 1035, November 1987, <<https://www.rfc-editor.org/info/rfc1035>>.
- [RFC2782] Gulbrandsen, A., Vixie, P., and L. Esibov, "A DNS RR for specifying the location of services (DNS SRV)", IETF, DOI 10.17487/RFC2782, RFC 2782, February 2000, <<https://www.rfc-editor.org/info/rfc2782>>.
- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", IETF, DOI 10.17487/RFC2119, BCP 14, RFC 2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", IETF, DOI 10.17487/RFC8174, BCP 14, RFC 8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.
- [RFC7208] Kitterman, S., "Sender Policy Framework (SPF) for Authorizing Use of Domains in Email, Version 1", IETF, DOI 10.17487/RFC7208, RFC 7208, April 2014, <<https://www.rfc-editor.org/info/rfc7208>>.
- [RFC6749] Hardt, D., "The OAuth 2.0 Authorization Framework", IETF, DOI 10.17487/RFC6749, RFC 6749, October 2012, <<https://www.rfc-editor.org/info/rfc6749>>.
- [RFC3597] Gustafsson, A., "Handling of Unknown DNS Resource Record (RR) Types", IETF, DOI 10.17487/RFC3597, RFC 3597, September 2003, <<https://www.rfc-editor.org/info/rfc3597>>.
- [RFC8259] Bray, T., "The JavaScript Object Notation (JSON) Data Interchange Format", IETF, STD 90, DOI 10.17487/RFC8259, BCP 90, RFC 8259, December 2017, <<https://www.rfc-editor.org/info/rfc8259>>.
- [RFC8552] Crocker, D., "Scoped Interpretation of DNS Resource Records through "Underscored" Naming of Attribute Leaves", IETF, DOI 10.17487/RFC8552, BCP 222, RFC 8552, March 2019, <<https://www.rfc-editor.org/info/rfc8552>>.

-
- [RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", IETF, STD 66, DOI 10.17487/RFC3986, BCP 66, RFC 3986, January 2005, <<https://www.rfc-editor.org/info/rfc3986>>.
- [RFC7518] Jones, M., "JSON Web Algorithms (JWA)", IETF, DOI 10.17487/RFC7518, RFC 7518, May 2015, <<https://www.rfc-editor.org/info/rfc7518>>.
- [RFC6335] Cotton, M., Eggert, L., Touch, J., Westerlund, M., and S. Cheshire, "Internet Assigned Numbers Authority (IANA) Procedures for the Management of the Service Name and Transport Protocol Port Number Registry", IETF, DOI 10.17487/RFC6335, BCP 165, RFC 6335, August 2011, <<https://www.rfc-editor.org/info/rfc6335>>.
- [RFC6895] 3rd, D. E., "Domain Name System (DNS) IANA Considerations", IETF, DOI 10.17487/RFC6895, BCP 42, RFC 6895, April 2013, <<https://www.rfc-editor.org/info/rfc6895>>.
- [RFC6570] Gregorio, J., Fielding, R., Hadley, M., Nottingham, M., and D. Orchard, "URI Template", IETF, DOI 10.17487/RFC6570, RFC 6570, March 2012, <<https://www.rfc-editor.org/info/rfc6570>>.
- [RFC5234] Overell, P. and D. Crocker, "Augmented BNF for Syntax Specifications: ABNF", IETF, STD 68, DOI 10.17487/RFC5234, BCP 68, RFC 5234, January 2008, <<https://www.rfc-editor.org/info/rfc5234>>.
- [RFC4648] Josefsson, S., "The Base16, Base32, and Base64 Data Encodings", IETF, DOI 10.17487/RFC4648, RFC 4648, October 2006, <<https://www.rfc-editor.org/info/rfc4648>>.
- [RFC5891] Klensin, J., "Internationalized Domain Names in Applications (IDNA): Protocol", IETF, DOI 10.17487/RFC5891, RFC 5891, August 2010, <<https://www.rfc-editor.org/info/rfc5891>>.
- [RFC5890] Klensin, J., "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework", IETF, DOI 10.17487/RFC5890, RFC 5890, August 2010, <<https://www.rfc-editor.org/info/rfc5890>>.
- [RFC9839] Bray, T. and P. Hoffman, "Unicode Character Repertoire Subsets", IETF, DOI 10.17487/RFC9839, RFC 9839, August 2025, <<https://www.rfc-editor.org/info/rfc9839>>.
- [RFC8126] Cotton, M., Leiba, B., and T. Narten, "Guidelines for Writing an IANA Considerations Section in RFCs", IETF, DOI 10.17487/RFC8126, BCP 26, RFC 8126, June 2017, <<https://www.rfc-editor.org/info/rfc8126>>.
- [RFC9499] Hoffman, P. and K. Fujiwara, "DNS Terminology", IETF, DOI 10.17487/RFC9499, BCP 219, RFC 9499, March 2024, <<https://www.rfc-editor.org/info/rfc9499>>.
-

- [RFC6838]** Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", IETF, DOI 10.17487/RFC6838, BCP 13, RFC 6838, January 2013, <<https://www.rfc-editor.org/info/rfc6838>>.
- [RFC9110]** Fielding, R., Nottingham, M., and J. Reschke, "HTTP Semantics", IETF, STD 97, DOI 10.17487/RFC9110, BCP 97, RFC 9110, June 2022, <<https://www.rfc-editor.org/info/rfc9110>>.
- [RFC2181]** Elz, R. and R. Bush, "Clarifications to the DNS Specification", IETF, DOI 10.17487/RFC2181, RFC 2181, July 1997, <<https://www.rfc-editor.org/info/rfc2181>>.

Informative References

- [RFC6839]** Hansen, T. and A. Melnikov, "Additional Media Type Structured Syntax Suffixes", IETF, DOI 10.17487/RFC6839, RFC 6839, January 2013, <<https://www.rfc-editor.org/info/rfc6839>>.
- [DC-SPEC]** Blinn, A., Kerola, S., and P. Kowalik, "Domain Connect Specification, Version 2.3 Rev. 67", 2 March 2025, <<https://github.com/Domain-Connect/spec/blob/rev.67/Domain%20Connect%20Spec%20Draft.adoc>>.

Appendix A. Examples

A.1. Example Template

EXAMPLE: Full template example

```
{
  "providerId": "example.com",
  "providerName": "Example Web Hosting",
  "serviceId": "hosting",
  "serviceName": "WordPress by example.com",
  "version": 1,
  "logoUrl": "https://www.example.com/images/billthecat.jpg",
  "description": "This connects your domain to our web hosting",
  "records": [
    {
      "type": "A",
      "groupId": "service",
      "host": "www",
      "pointsTo": "%var1%",
      "ttl": 600
    },
    {
      "type": "A",
      "groupId": "service",
      "host": "m",
      "pointsTo": "%var2%",
      "ttl": 600
    },
    {
      "type": "CNAME",
      "groupId": "service",
      "host": "webmail",
      "pointsTo": "%var3%",
      "ttl": 600
    },
    {
      "type": "TXT",
      "groupId": "verification",
      "host": "example",
      "ttl": 600,
      "data": "%var4%"
    }
  ]
}
```

A.2. Example Records: Single static host record

Consider a template for setting a single host record. The records section of the template would have a single record of type "A" and could have a value of:

EXAMPLE: Single static host record example

```
[{  
  "type": "A",  
  "host": "www",  
  "pointsTo": "192.0.2.1",  
  "ttl": 600  
}]
```

This would have no variable substitution and the application of this template to a domain would simply set the host name "www" to the IP address "192.0.2.1"

A.3. Example Records: Single variable host record for A

In the case of a template for setting a single host record from a variable, the template would have a single record of type "A" and could have a value of:

EXAMPLE: Single variable host record for A example

```
[{  
  "type": "A",  
  "host": "@",  
  "pointsTo": "198.51.100.%srv%",  
  "ttl": 600  
}]
```

A query string with a key/value pair of

```
srv=2
```

would cause the application of this template to a domain to set the host name for the apex A record to the IP address "198.51.100.2" with a TTL of 600

A.4. Example Records: Generic Record Type CAA

This example shows how to include a set of Generic Record Types on an example of CAA records:

EXAMPLE: Generic Record Type CAA example

```
[
  {
    "type": "CAA",
    "host": "@",
    "data": "0 issue \"ca1.example.net\"",
    "ttl": 1800
  },
  {
    "type": "CAA",
    "host": "@",
    "data": "0 issuewild \"ca2.example.\",
    "ttl": 1800
  }
]
```

This would have no variable substitution and the application of this template to a domain would add 2 CAA records.

A.5. Example: Vendor-Specific Extensions and Critical Markers

This example shows a template using vendor-specific extensions at both the template and the record level (see [Section 6.1.1](#) and [Section 6.2.1](#)):

EXAMPLE: Template with vendor-specific extensions example

```
{
  "providerId": "example.com",
  "providerName": "Example Web Hosting",
  "serviceId": "cdn",
  "serviceName": "Example CDN",
  "version": 1,
  "vnd.example.contact": {
    "supportUrl": "https://support.example.com/cdn",
    "email": "support@example.com"
  },
  "vnd.example.lockRecords": true,
  "crit": ["vnd.example.lockRecords"],
  "records": [
    {
      "type": "A",
      "host": "www",
      "pointsTo": "%ip%",
      "ttl": 600,
      "vnd.example.proxied": true,
      "crit": ["vnd.example.proxied"]
    },
    {
      "type": "CNAME",
      "host": "static",
      "pointsTo": "cdn.example.com",
      "ttl": 600,
      "vnd.example.comment": "Static content"
    }
  ]
}
```

A.6. Example: Template application to DNS Zone and Conflict Resolution

Consider a DNS Zone before a template application:

```
$ORIGIN example.com.

@ 3600 IN SOA ns11.example.net. support.example.net. 2017050817 7200
1800 1209600 3600
@ 3600 IN NS ns11.example.net.
@ 3600 IN NS ns12.example.net.
@ 3600 IN A 192.0.2.1
@ 3600 IN A 192.0.2.2
@ 3600 IN AAAA 2001:db8:1234:0000:0000:0000:0000:0000
@ 3600 IN AAAA 2001:db8:1234:0000:0000:0000:0000:0001
@ 3600 IN MX 10 mx1.example.net.
@ 3600 IN MX 10 mx2.example.net.
@ 3600 IN TXT "v=spf1 a include:spf.example.org ~all"
www 3600 IN CNAME other.host.example.
```

Now application of the following template:

```
[
  {
    "type": "A",
    "host": "@",
    "pointsTo": "203.0.113.2",
    "ttl": "1800"
  },
  {
    "type": "A",
    "host": "www",
    "pointsTo": "203.0.113.2",
    "ttl": "1800"
  },
  {
    "type": "SPFM",
    "host": "@",
    "spfRules": "a include:spf.hoster.example"
  }
]
```

The following DNS Zone would be generated after the template is applied.

A following list of conflicts would be detected in this case:

```
@ 3600 IN A 192.0.2.1
@ 3600 IN A 192.0.2.2
@ 3600 IN AAAA 2001:db8:1234:0000:0000:0000:0000:0000
@ 3600 IN AAAA 2001:db8:1234:0000:0000:0000:0000:0001
www 3600 IN CNAME other.host.example.
```

After changes applied the zone would be updated to the following state. Note existing A and AAAA records removed due to the conflicts as well as SPF TXT record content merged with the existing entry.

```
$ORIGIN example.com.

@ 3600 IN SOA ns11.example.net. support.example.net. 2017050920 7200
1800 1209600 3600
@ 3600 IN NS ns11.example.net.
@ 3600 IN NS ns12.example.net.
@ 1800 IN A 203.0.113.2
@ 3600 IN MX 10 mx1.example.net.
@ 3600 IN MX 10 mx2.example.net.
@ 1800 IN TXT "v=spf1 a include:spf.example.org include:spf.hoster.ex
ample ~all"
www 1800 IN A 203.0.113.2
```

A.7. Example: SPF Record Merging

Consider a DNS Zone before a template application:

```
$ORIGIN example.com.
```

```
@ 3600 IN SOA ns11.example.net. support.example.net. 2017050817 7200
1800 1209600 3600
@ 3600 IN NS ns11.example.net.
@ 3600 IN NS ns12.example.net.
```

Now application of the following template of Mail service:

```
[
  {
    "type": "MX",
    "host": "@",
    "priority": "10",
    "pointsTo": "mx1.example.net",
    "ttl": "1800"
  },
  {
    "type": "MX",
    "host": "www",
    "priority": "10",
    "pointsTo": "mx2.example.net",
    "ttl": "1800"
  },
  {
    "type": "SPFM",
    "host": "@",
    "spfRules": "a include:spf.example.net"
  }
]
```

Expected result in the DNS Zone

```
$ORIGIN example.com.

@ 3600 IN SOA ns11.example.net. support.example.net. 2017050817 7200
1800 1209600 3600
@ 3600 IN NS ns11.example.net.
@ 3600 IN NS ns12.example.net.
@ 1800 IN MX 10 mx1.example.net.
www 1800 IN MX 10 mx2.example.net.
@ 3600 IN TXT "v=spf1 a include:spf.example.net ~all"
```

In the next step application of the following template of Newsletter service:

```
[
  {
    "type": "SPFM",
    "host": "@",
    "spfRules": "include:_spf.newsletter.example"
  }
]
```

Expected result in the DNS Zone. Note merged SPF entry.

```
$ORIGIN example.com.

@ 3600 IN SOA ns11.example.net. support.example.net. 2017050817 7200
1800 1209600 3600
@ 3600 IN NS ns11.example.net.
@ 3600 IN NS ns12.example.net.
@ 1800 IN MX 10 mx1.example.net.
www 1800 IN MX 10 mx2.example.net.
@ 3600 IN TXT "v=spf1 a include:spf.example.net include:_spf.newslett
er.
example ~all"
```

Authors' Addresses

P Kowalik

DENIC eG

Theodor-Stern-Kai 1

Frankfurt am Main

Germany

Email: pawel.kowalik@denic.de

URI: <https://denic.de>

A Blinn

Email: arnold@arnoldblinn.com

J Kolker

GoDaddy Inc.

14455 N. Hayden Rd. #219

Scottsdale,

United States of America

Email: jkolker@godaddy.com

URI: <https://www.godaddy.com>

S Kerola

Cloudflare, Inc.

101 Townsend St

San Francisco,

United States of America

Email: kerolasa@cloudflare.comURI: <https://cloudflare.com>